UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

CS 598: Scalable ML Overview

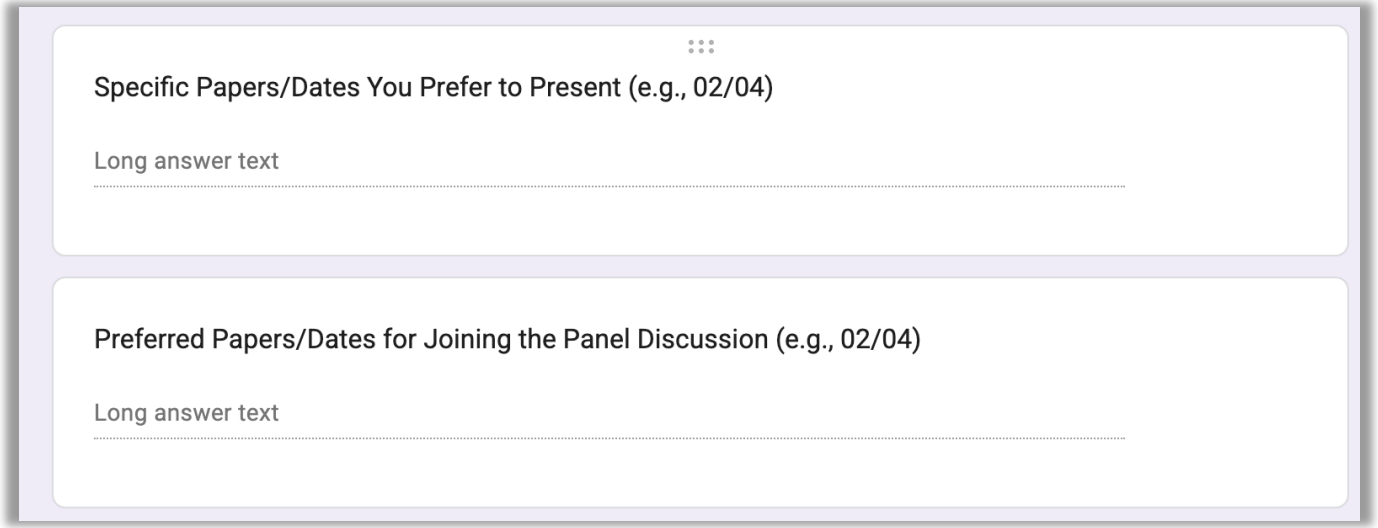
Fan Lai

The Grainger College of Engineering

Materials with thanks to Minjia Zhang, Arvind Krishnamurthy, and many colleagues

Reminder

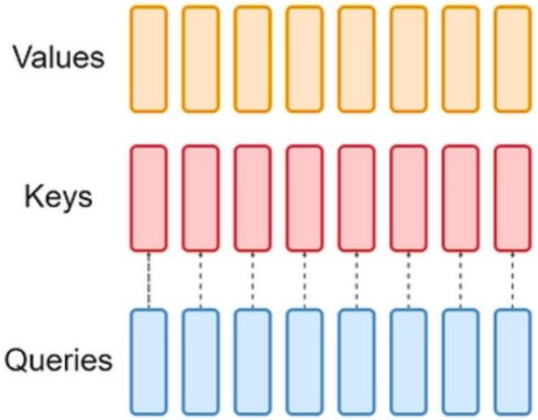
- **Team registration**
  - ***Five*** members in a team by Feb 1st



- **No class next Wednesday (02/04; for proposal Q&A in SC 3128)**
  - (Optional) stop by with your teammates during class hours
- **First paper summary due next Thursday 11:59 PM**

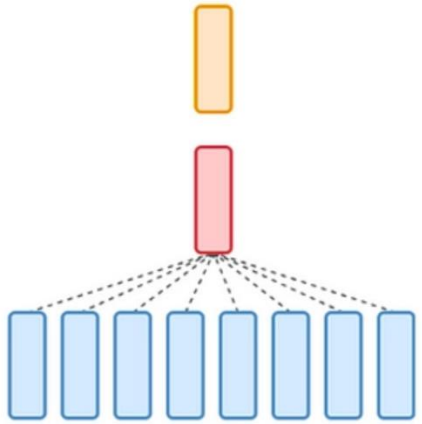
Recall: Multi-Head/Query Attention

Multi-head Attention



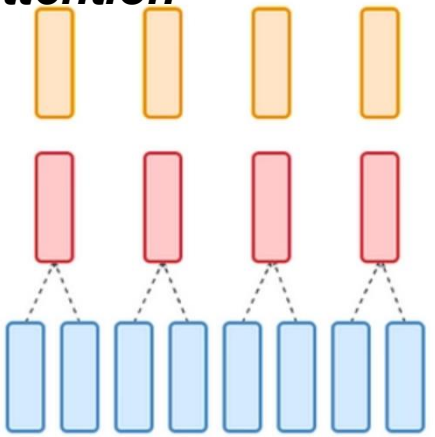
Each head digests QKV separately

Multi-Query Attention



Share single key and value heads across query heads

Grouped-Query Attention



Share single key and value heads for each group of query heads

Today

- **Distributed ML**
  - **Problem**
  - **Data Parallelism (DP)**
  - **Model Parallelism**

ML/DL Training Problem Definition Recap

- Given model f, data set $\{xi, y_i\}_{i=1}^{N}$
- Minimize the loss between predicted labels and true labels:

  Min $\frac{1}{N}\sum_{i=1}^{N} loss(f(x_i, y_i))$

- Common loss function
  - Cross-entropy, MSE (mean squared error)

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

MSE = mean squared error

n = number of data points

Y_i = observed values

$\hat{Y}_i$ = predicted values

ML/DL Training Problem Definition Recap

- Given model f, data set $\{xi, y_i\}_{i=1}^{N}$
- Minimize the loss between predicted labels and true labels:

  Min $\frac{1}{N}\sum_{i=1}^{N} loss(f(x_i, y_i))$

- Common loss function
  - Cross-entropy, MSE (mean squared error)
- Common way to solve the minimization problem
  - Adaptive learning rates optimizers (e.g., Adam)
  - Stochastic gradient descent (SGD)

Gradient Descent

- Model f_w is parameterized by weight w
- $\eta > 0$ is the learning rate

For t = 1 to T (iters)

Backward pass

Forward pass

$\Delta w = \eta \times \frac{1}{N} \sum_{i=1}^{N} \nabla \left(loss\left(f_w(x_i, y_i) \right) \right)$ // compute derivative and update

w -= Δw // apply update

End

Accelerating Gradient Descent

- Model f_w is parameterized by weight w
- $\eta > 0$ is the learning rate

Can we accelerate it?

For t = 1 to T (iters)

$\Delta w = \eta \times \frac{1}{N} \sum_{i=1}^{N} \nabla \left(loss\left(f_w(x_i, y_i) \right) \right)$ // compute derivative and update

w -= Δw // apply update

End

Accelerating Gradient Descent

- Model f_w is parameterized by weight w
- $\eta > 0$ is the learning rate

Increase the batch size increases throughput but eventually bounded by single GPU compute

For t = 1 to T (iters)

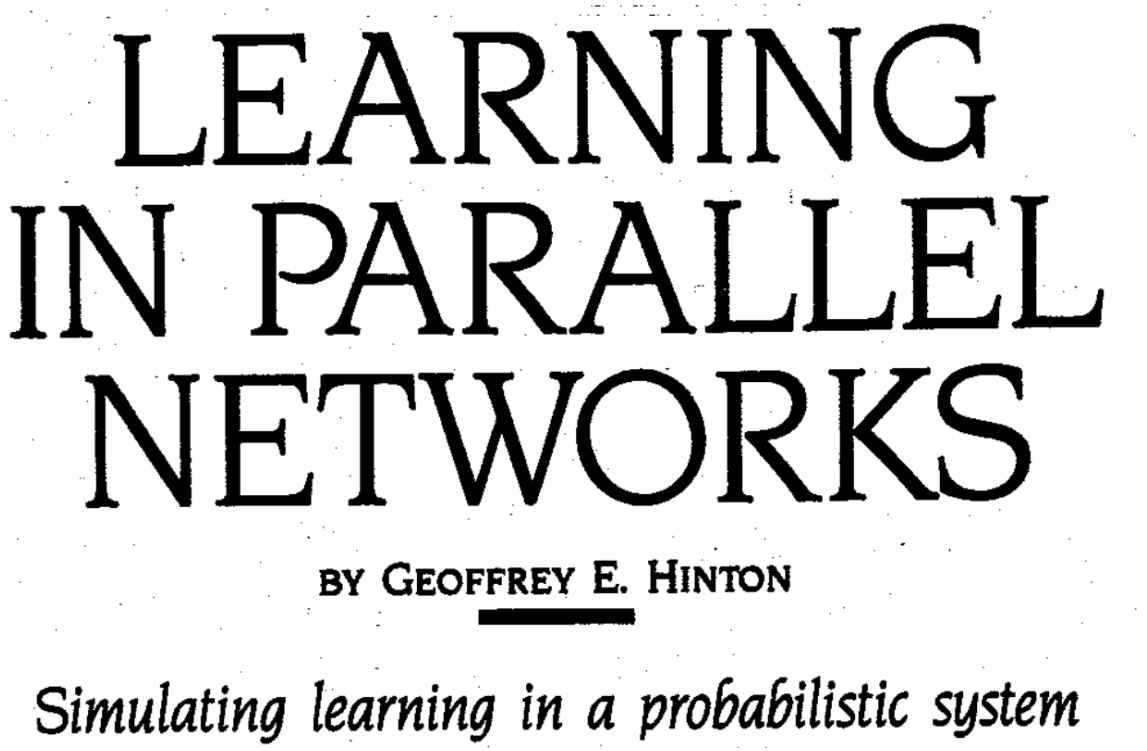
$$\Delta w = \eta \times \frac{1}{N} \sum_{i=1}^{N} \nabla \left(loss\left(f_w(x_i, y_i) \right) \right)$$ // compute derivative and update

w -= Δw // apply update

End

Distributed DL History

1985



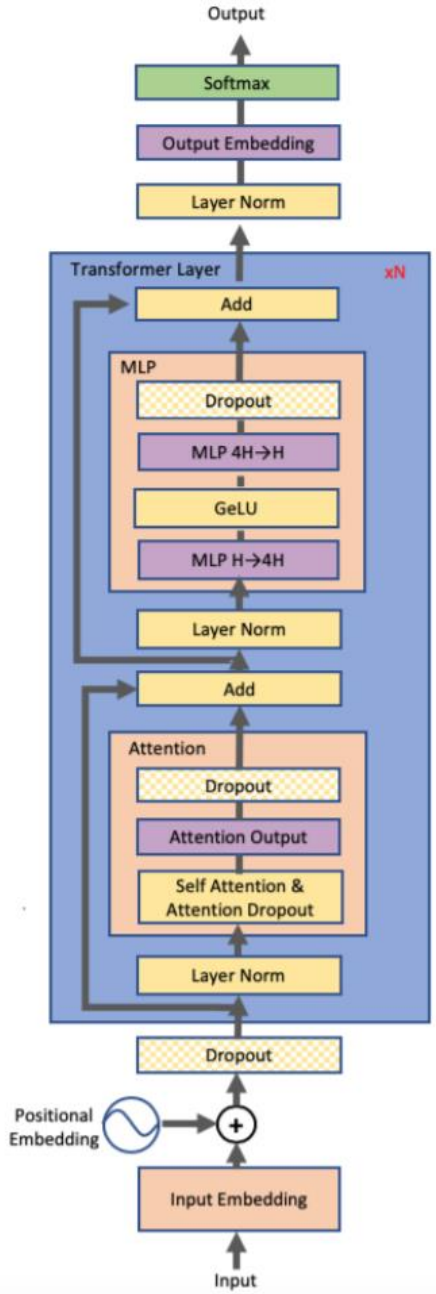
LEARNING
IN PARALLEL
NETWORKS

BY GEOFFREY E. HINTON

Simulating learning in a probabilistic system

Large Model Training Challenges

	Bert-Large	GPT-2	Turing 17.2 NLG	GPT-3
Parameters	0.32B	1.5B	17.2B	175B
Layers	24	48	78	96
Hidden Dimension	1024	1600	4256	12288
Relative Computation	1x	4.7x	54x	547x
Memory Footprint	5.12GB	24GB	275GB	2800GB

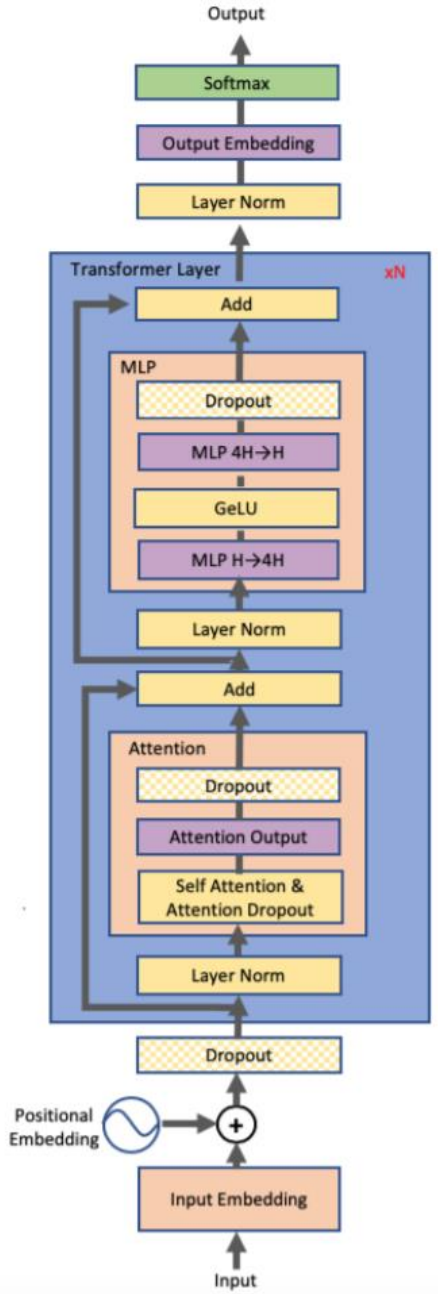


Large Model Training Challenges

	Bert-Large	GPT-2	Turing 17.2 NLG	GPT-3
Parameters	0.32B	1.5B	17.2B	175B
Layers	24	48	78	96
Hidden Dimension	1024	1600	4256	12288
Relative Computation	1x	4.7x	**54x**	**547x**
Memory Footprint	5.12GB	24GB	**275GB**	**2800GB**

Out of Memory

NVIDIA V100 GPU memory capacity: 16G/32G
NVIDIA A100 GPU memory capacity: 40G/80G



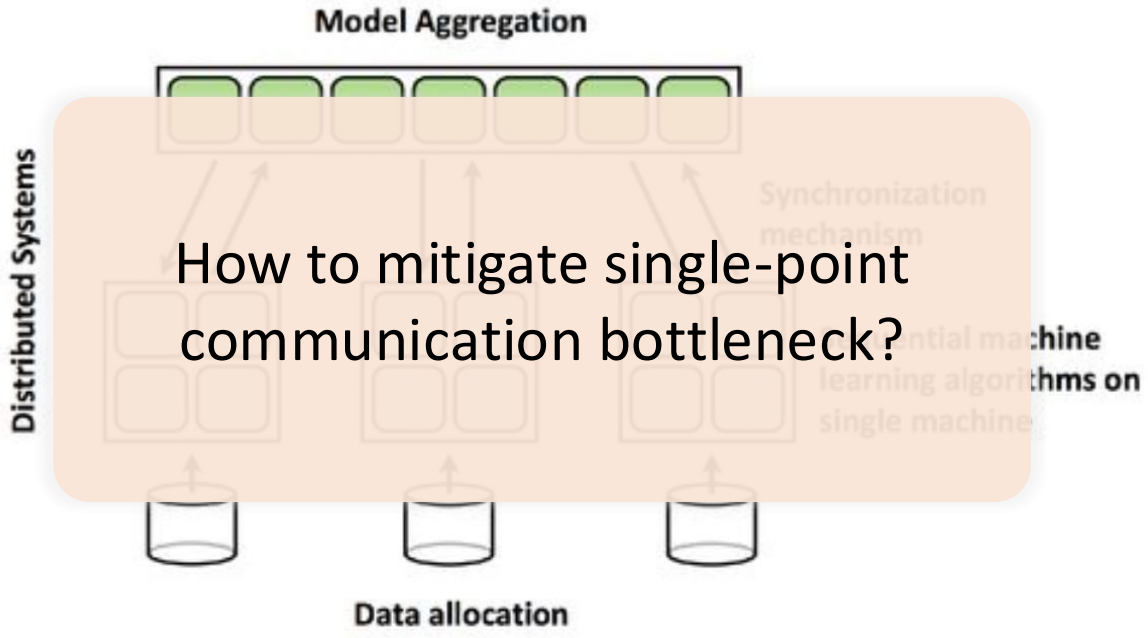
Today

- **Distributed Training**
  - **Problem**
  - **Data Parallelism (DP)**
  - **Model Parallelism**

Two Solutions

- (Centralized) Parameter Server
- (Decentralized) AllReduce
- Key assumption:
  - The model can fit into a (GPU) worker memory so we can create many model replicas

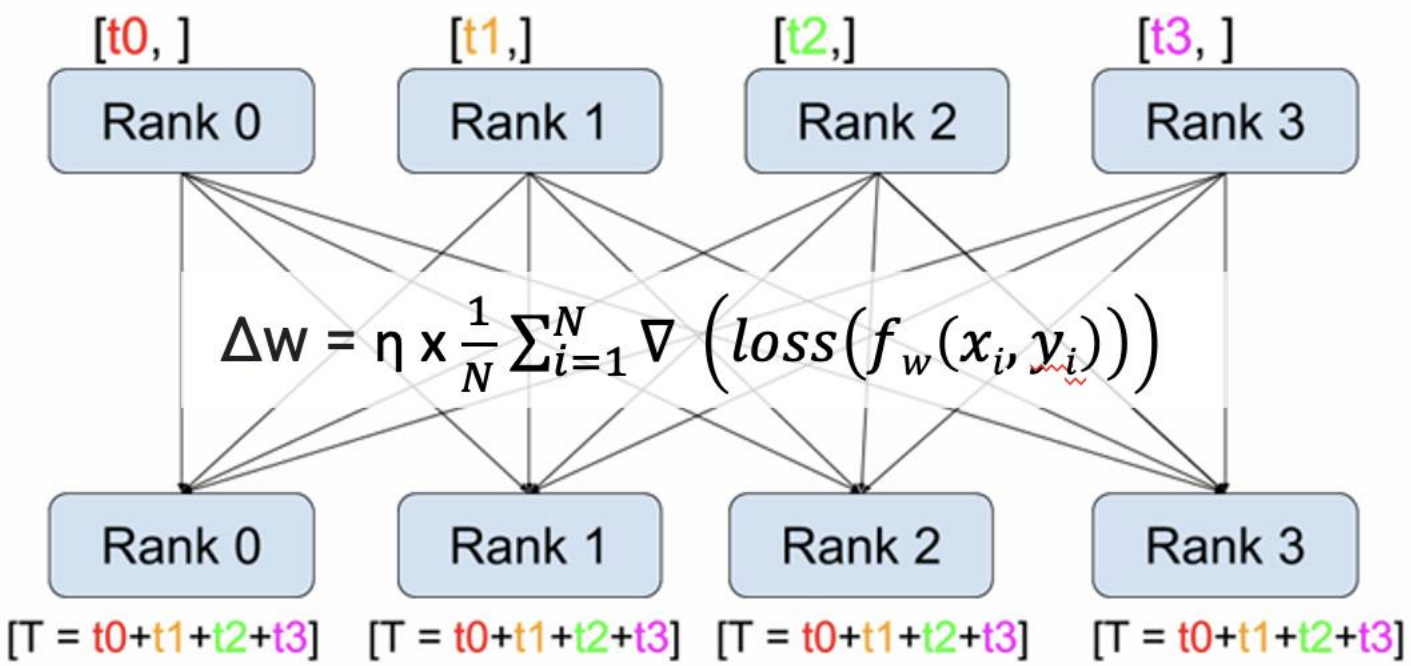
Data Parallelism with Parameter Server



How to mitigate single-point communication bottleneck?

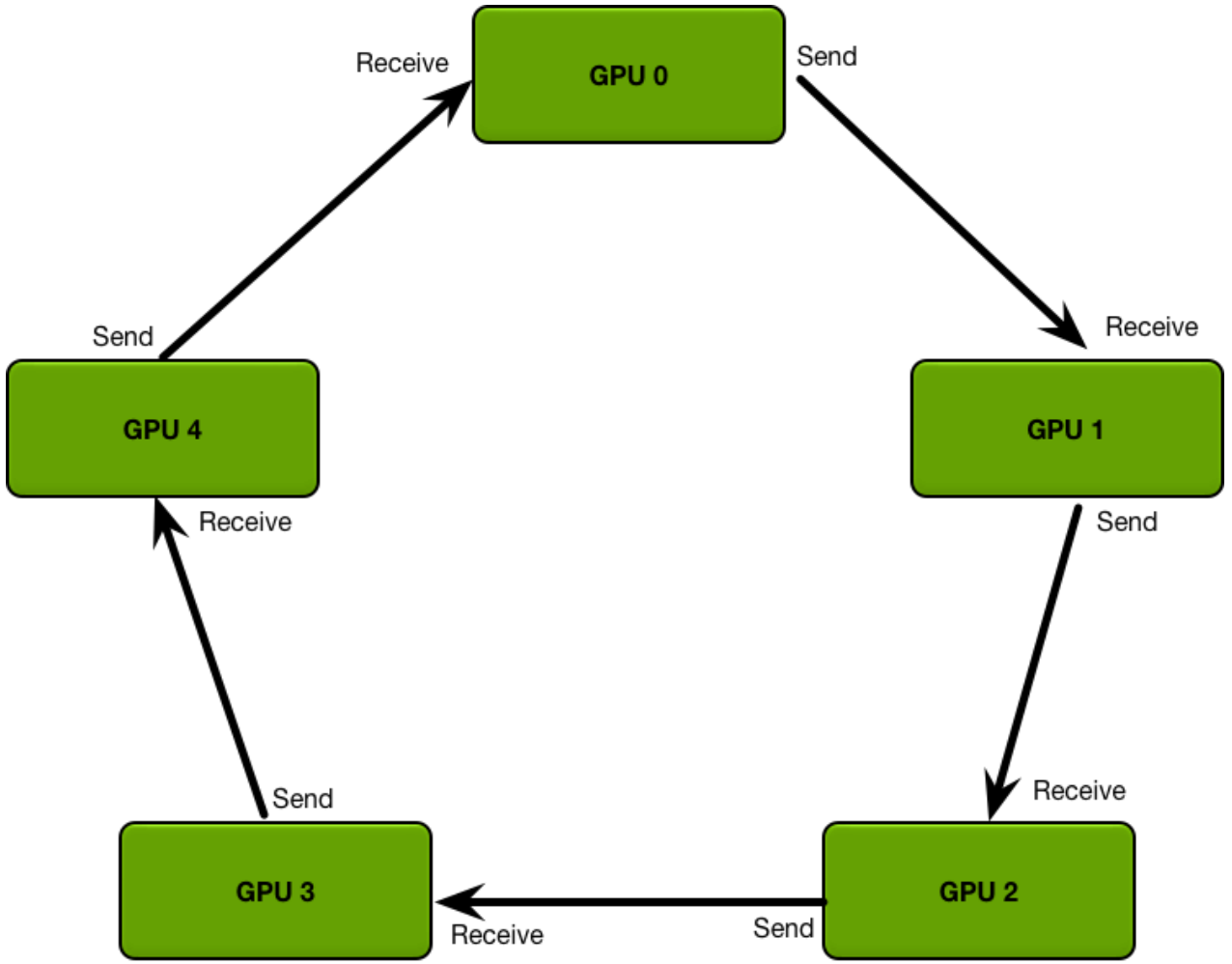
1. Partition the training data
2. Parallel training on different machines
3. Synchronize the local updates
4. Refresh local model with new parameters, then go to 2

Data Parallelism with AllReduce



Each worker acts as the parameter server too!

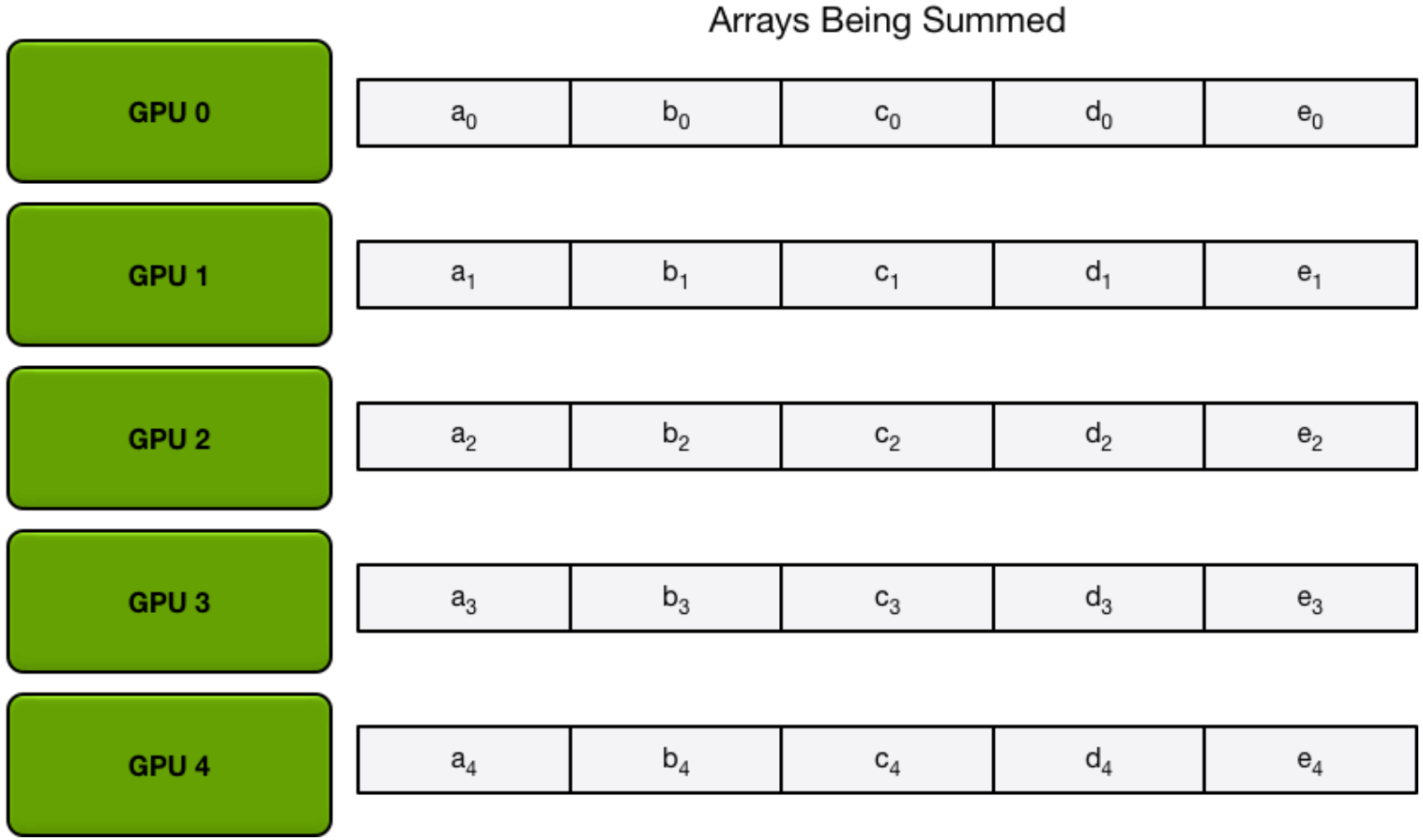
Data Parallelism with Ring-based AllReduce



GPUs arranged in a logical ring

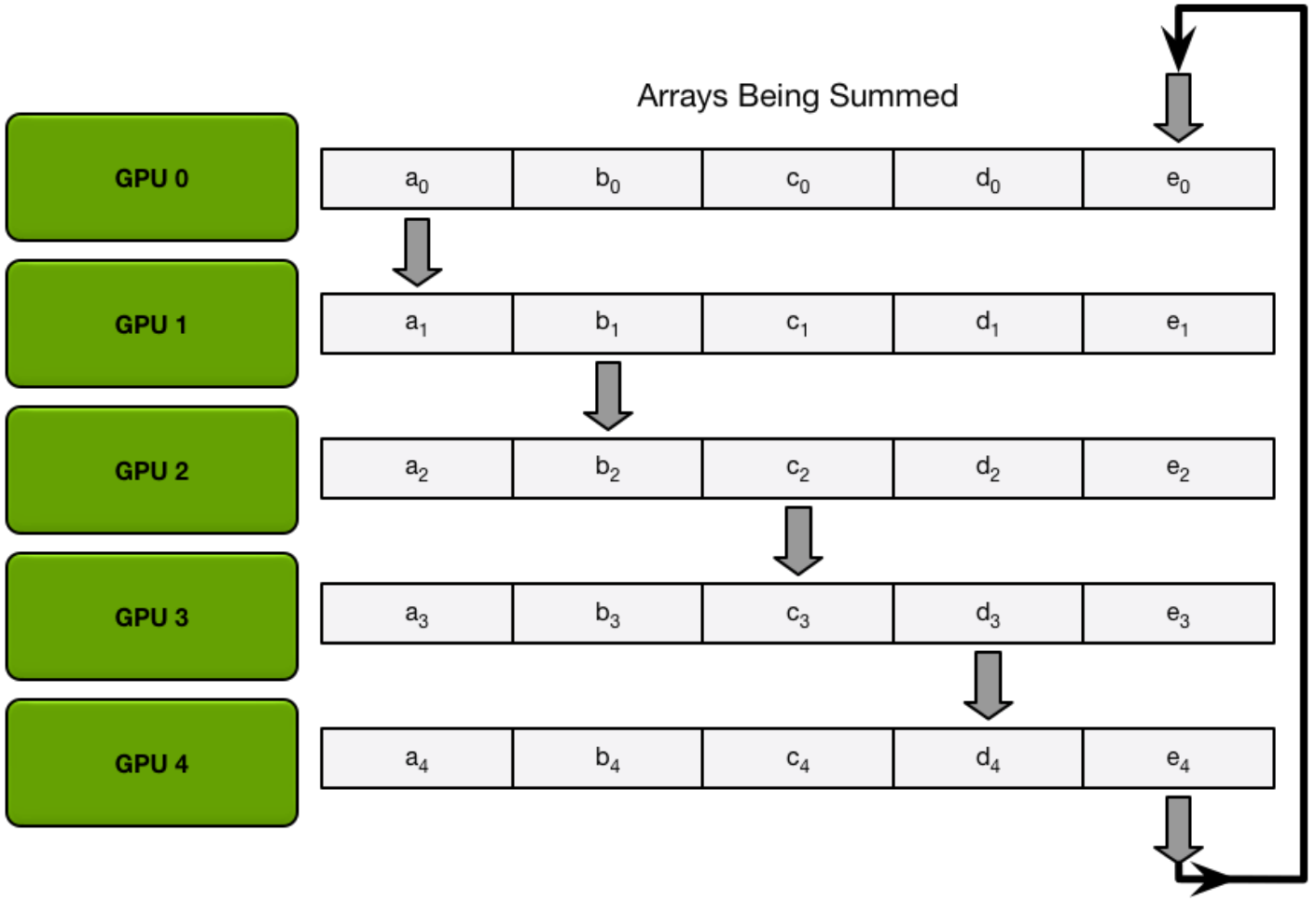
Two phases:
Scatter-reduce
All-gather

Phase 1: Reduce-Scatter



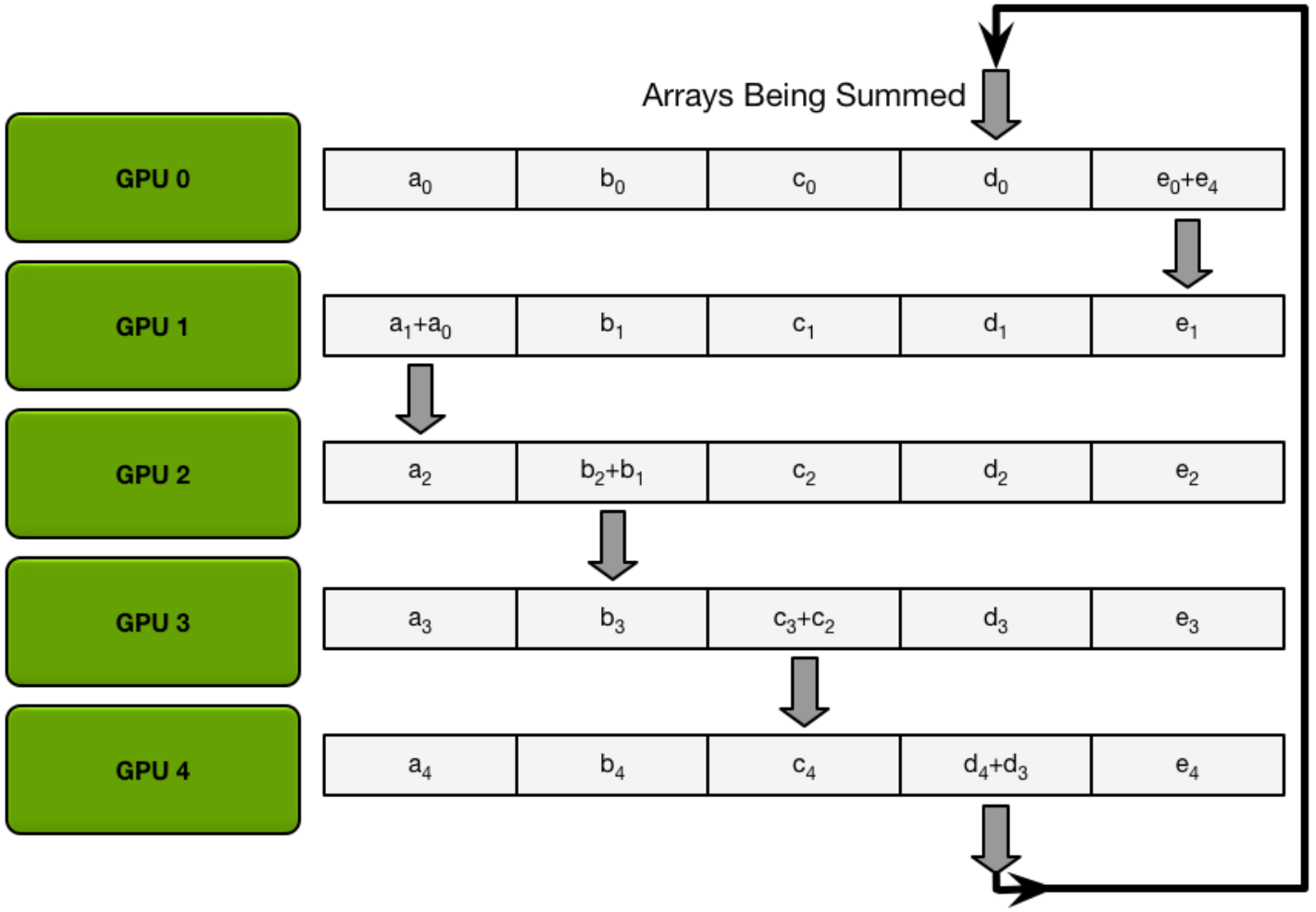
Partitioning of an array into N chunks

Phase 1: Reduce-Scatter



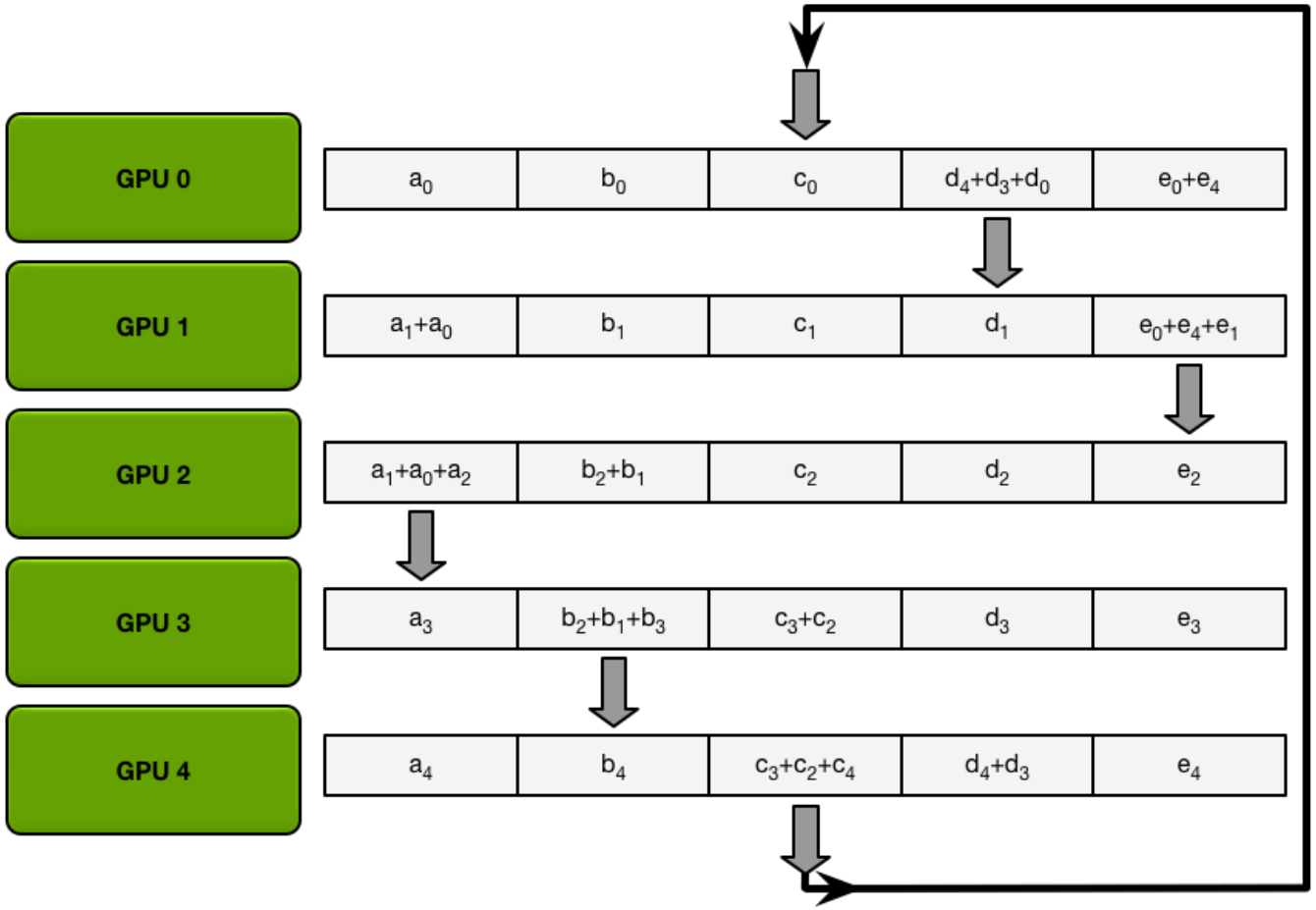
Data transfers in the 1st iteration of scatter-reduce

Phase 1: Reduce-Scatter



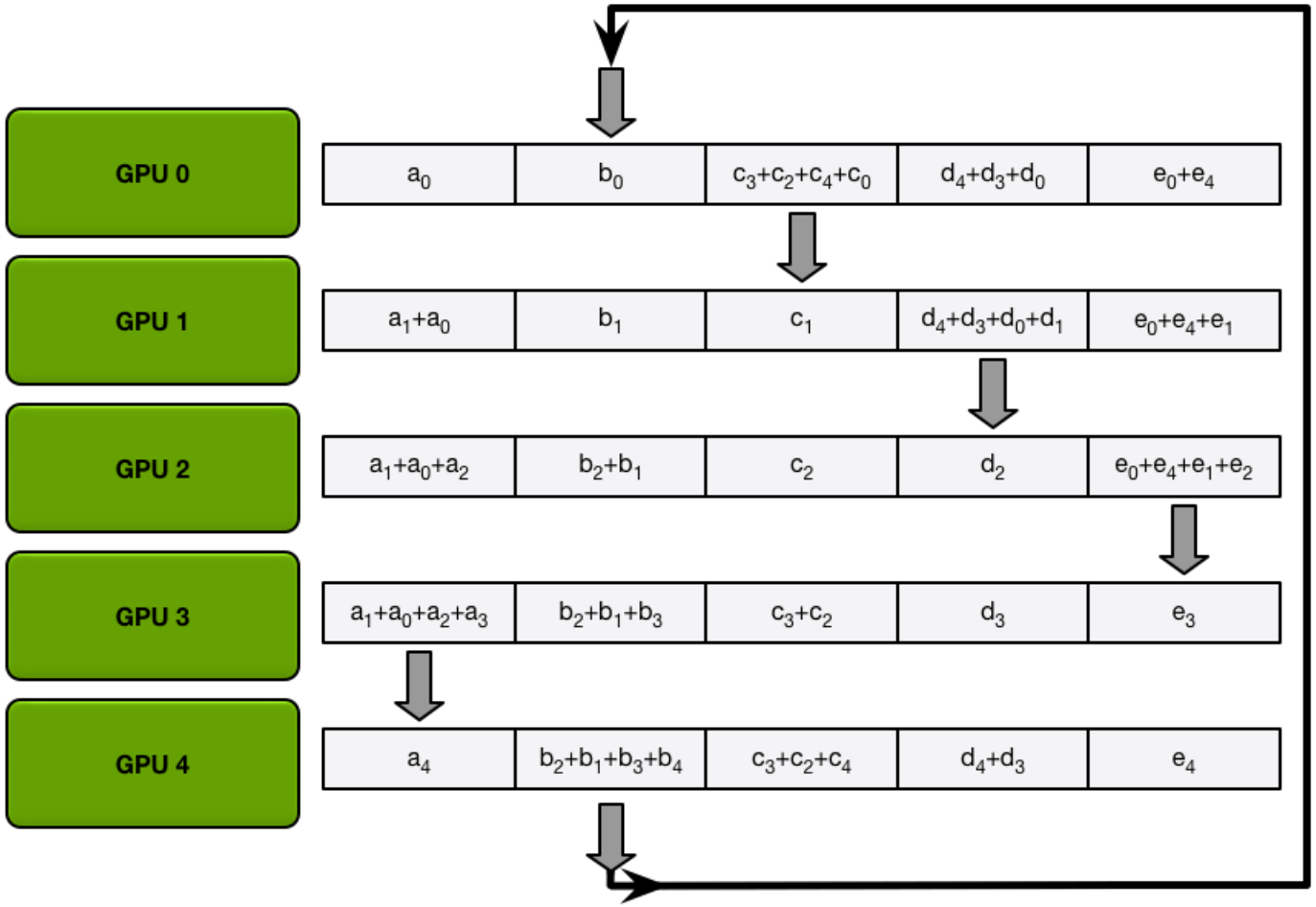
Intermediate sums after the first iteration of scatter-reduce is complete

Phase 1: Reduce-Scatter



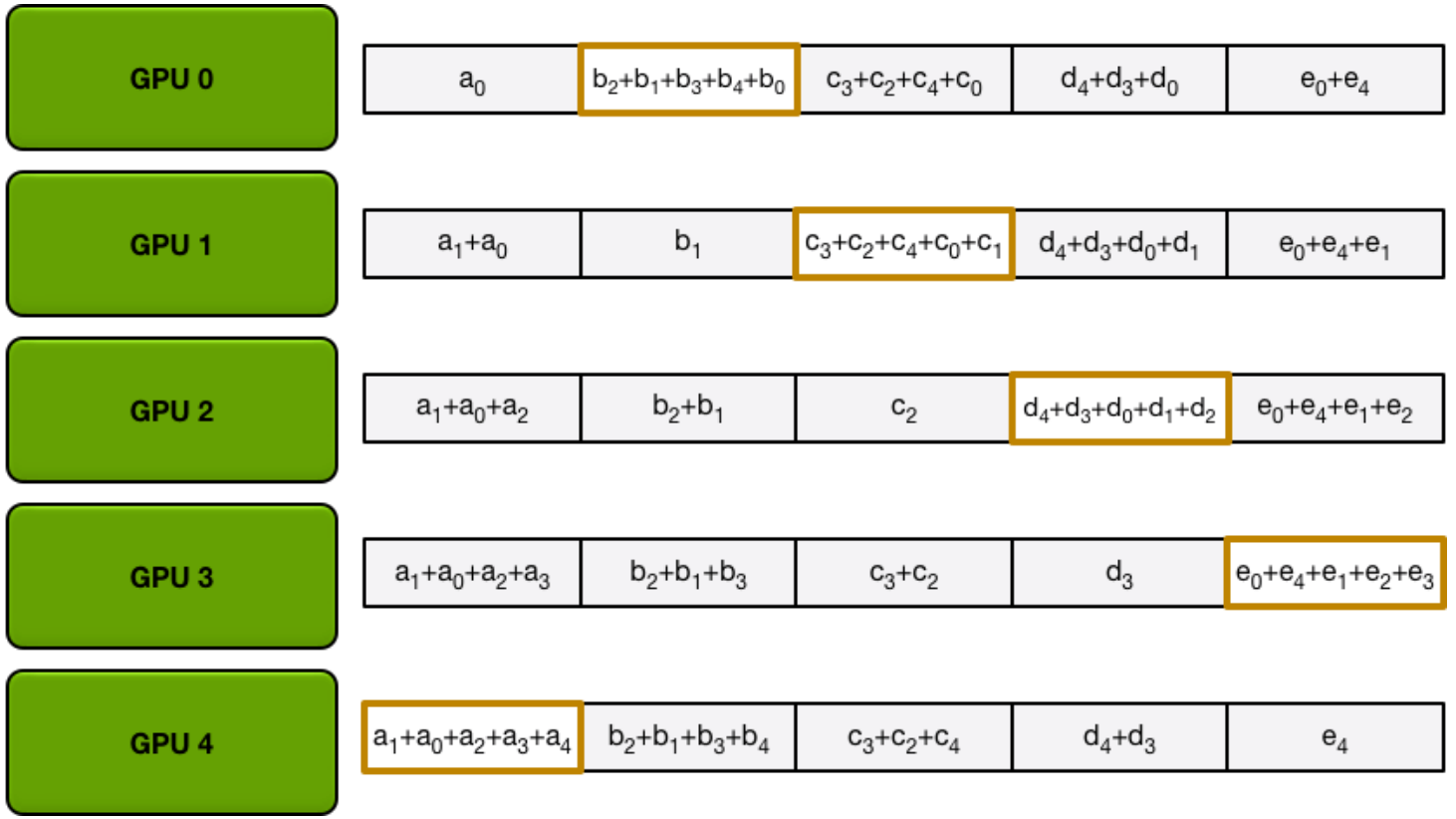
Reduce-scatter data transfer (iteration 3)

Phase 1: Reduce-Scatter



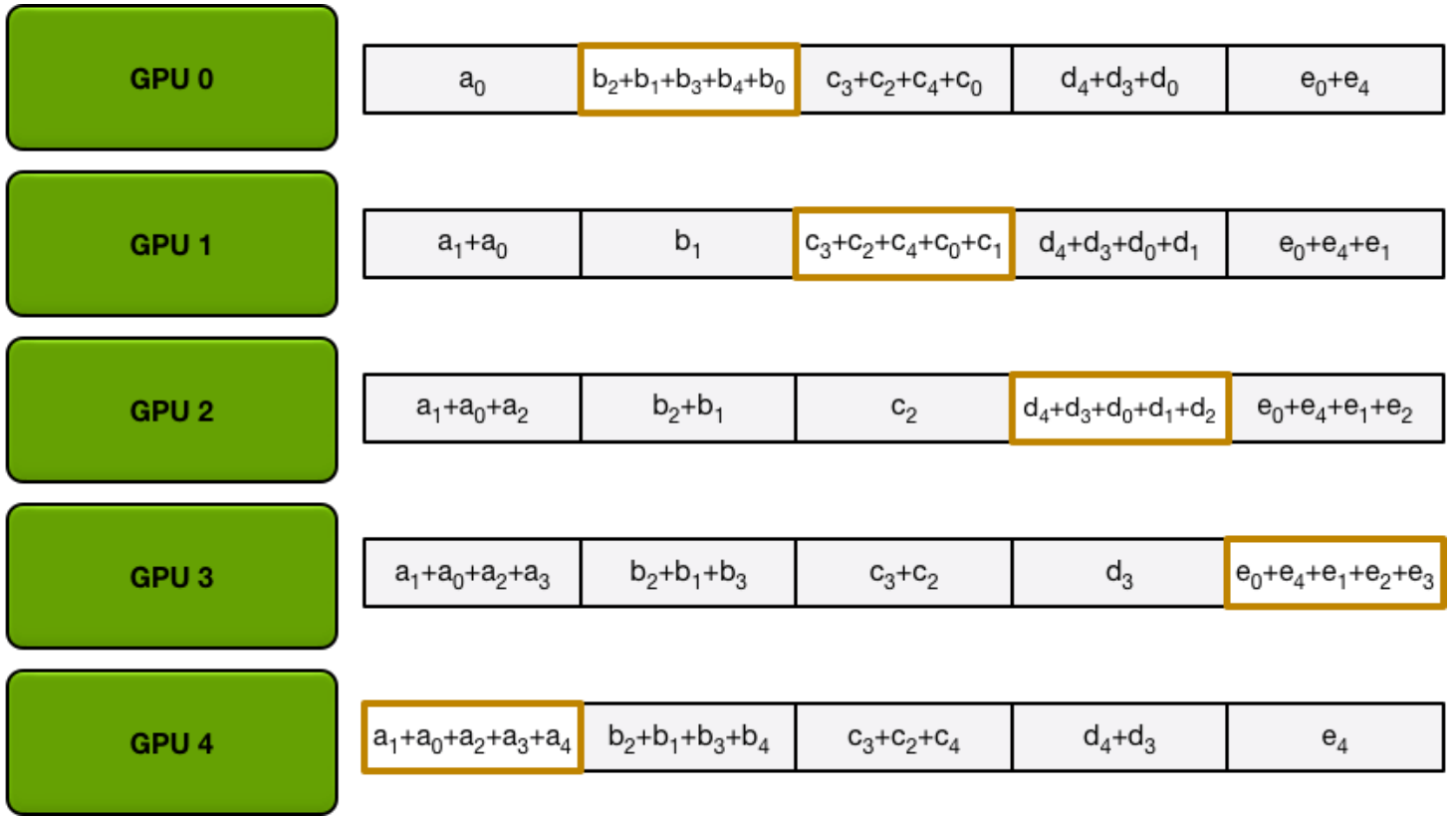
Reduce-scatter data transfer (iteration 4)

Phase 1: Reduce-Scatter



Reduce-scatter data transfer (after iteration 4)

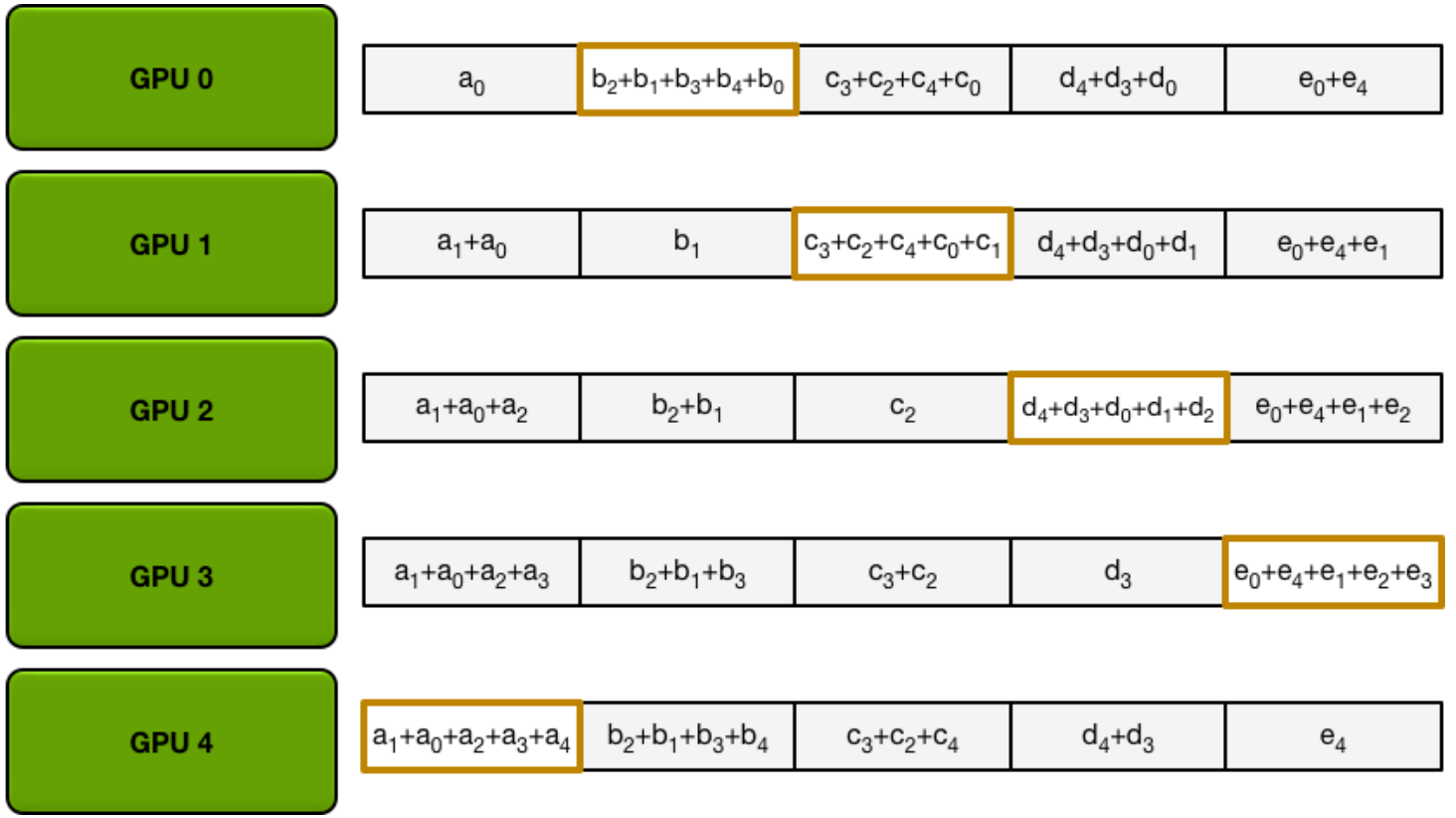
Phase 1: Reduce-Scatter



Question: What is the complexity of reduce-scatter?

Reduce-scatter data transfer (after iteration 4)

Phase 1: Reduce-Scatter



Reduce-scatter data transfer (after iteration 4)

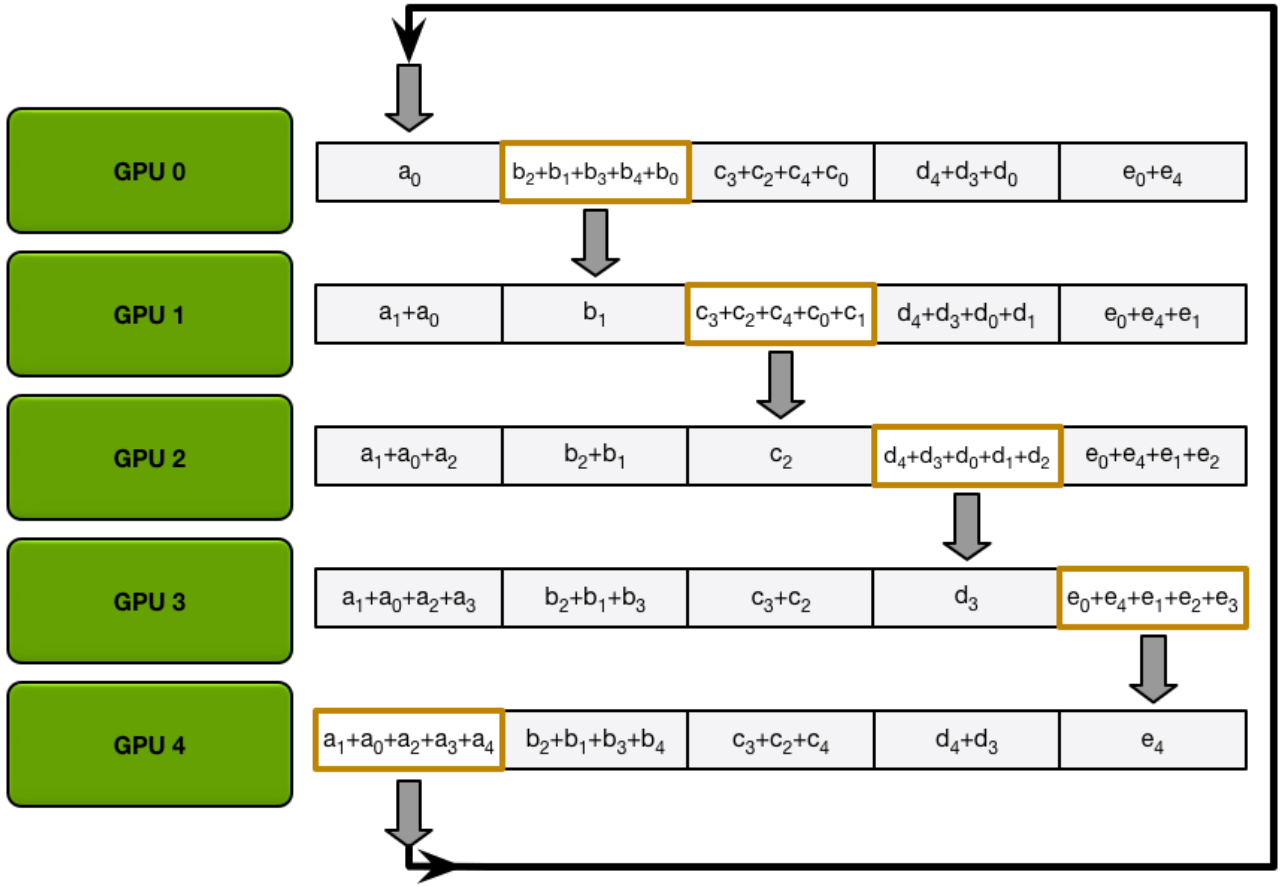
Question: What is the complexity of reduce-scatter?

- N: Size of data
- P: # of workers

$O(N/P * (P - 1))$

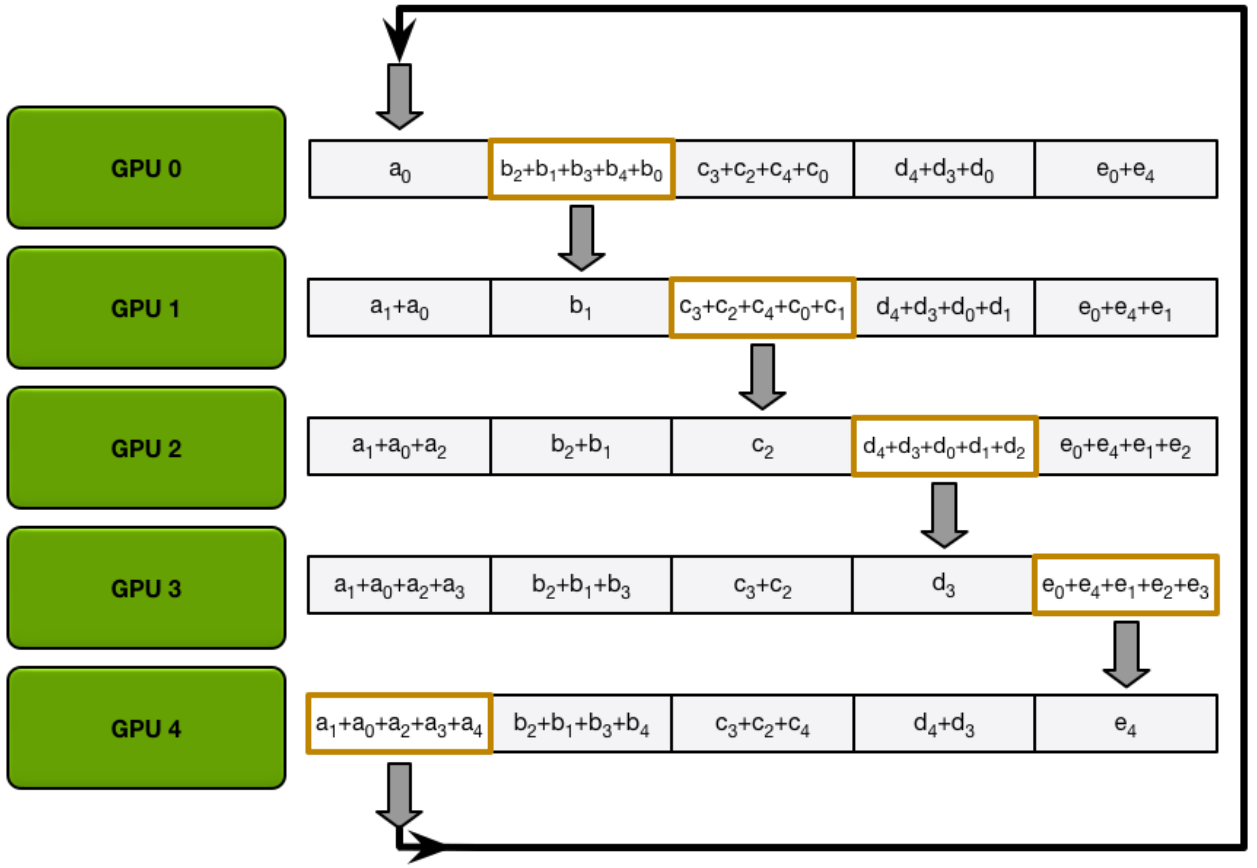
Chunk size

Phase 2: Allgather



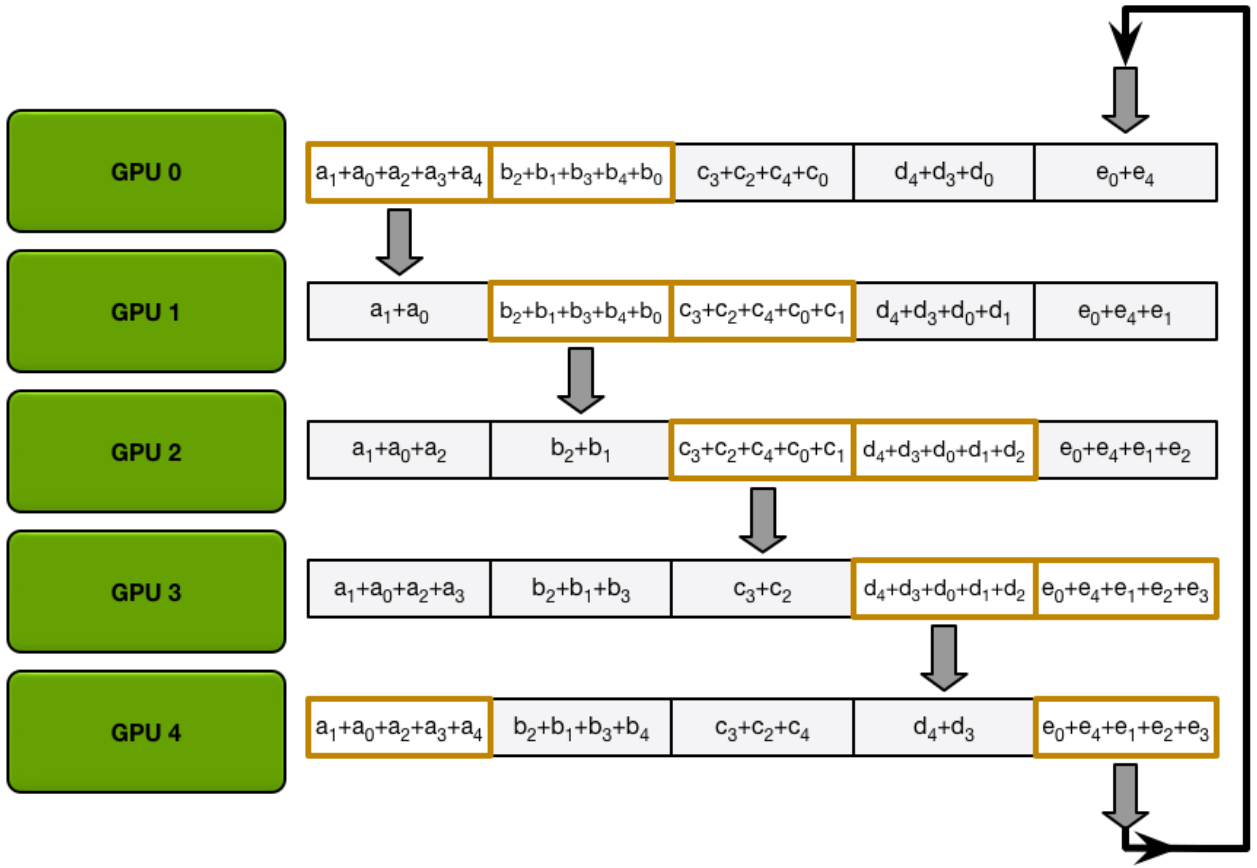
All processes can obtain the complete array by circulating again without reduction operations.

Phase 2: Allgather



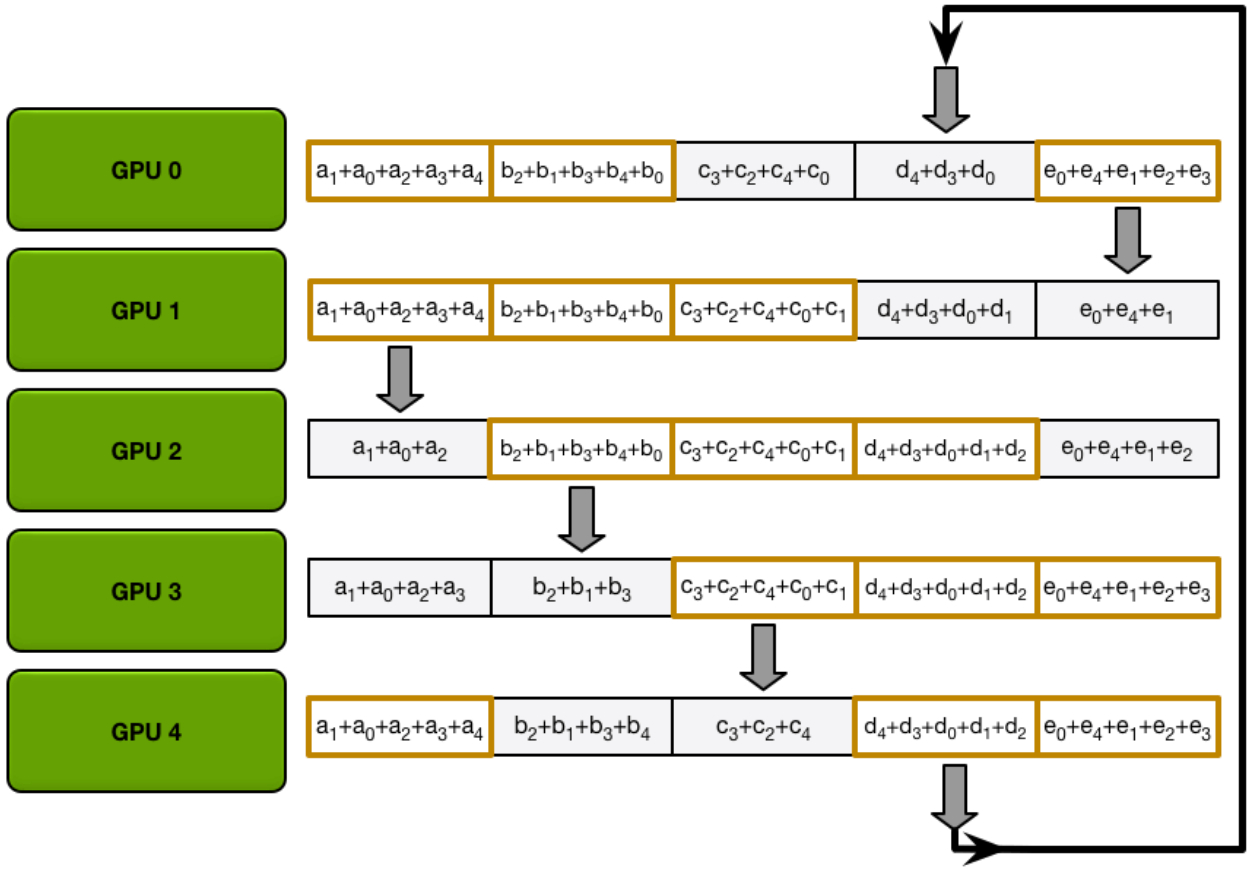
Allgather data transfer (iteration 1)

Phase 2: Allgather



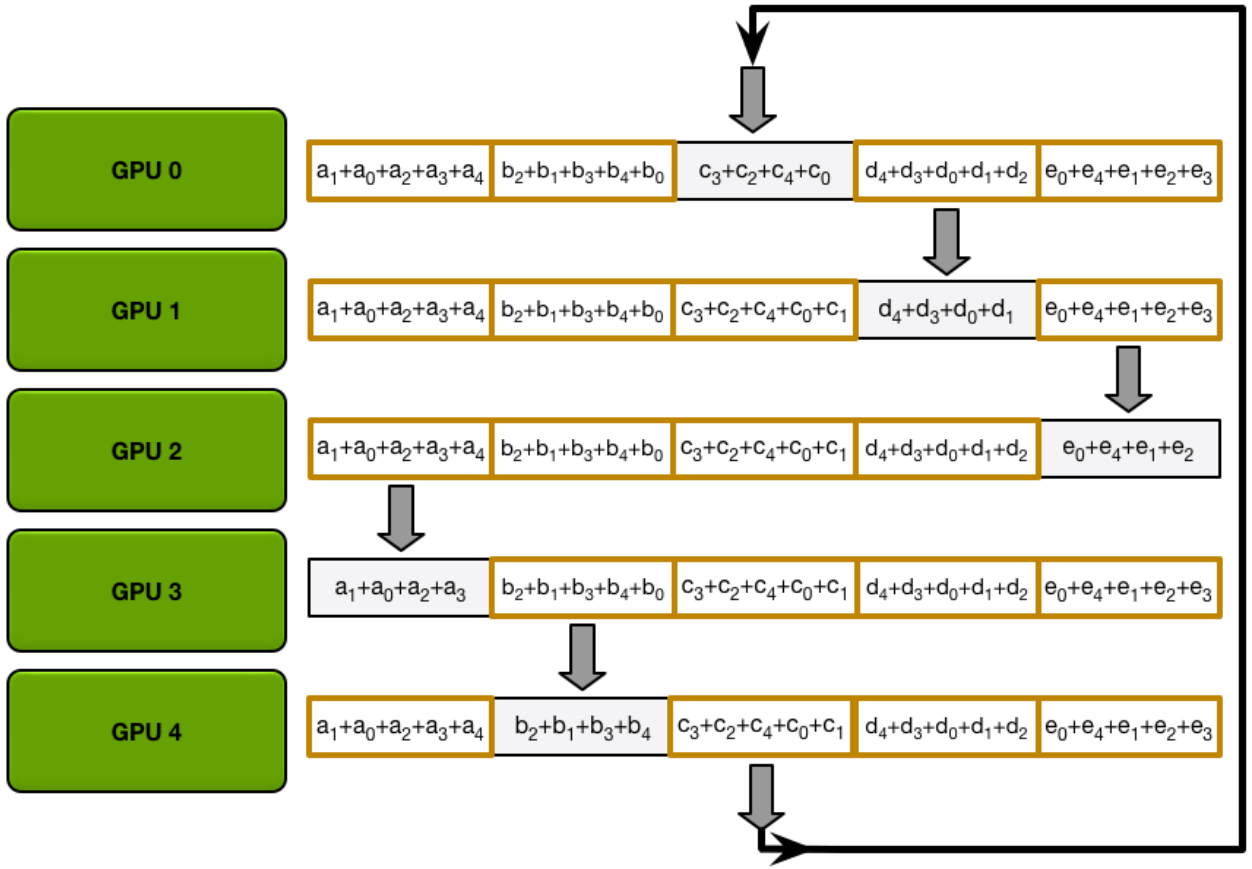
Allgather data transfer (iteration 2)

Phase 2: Allgather



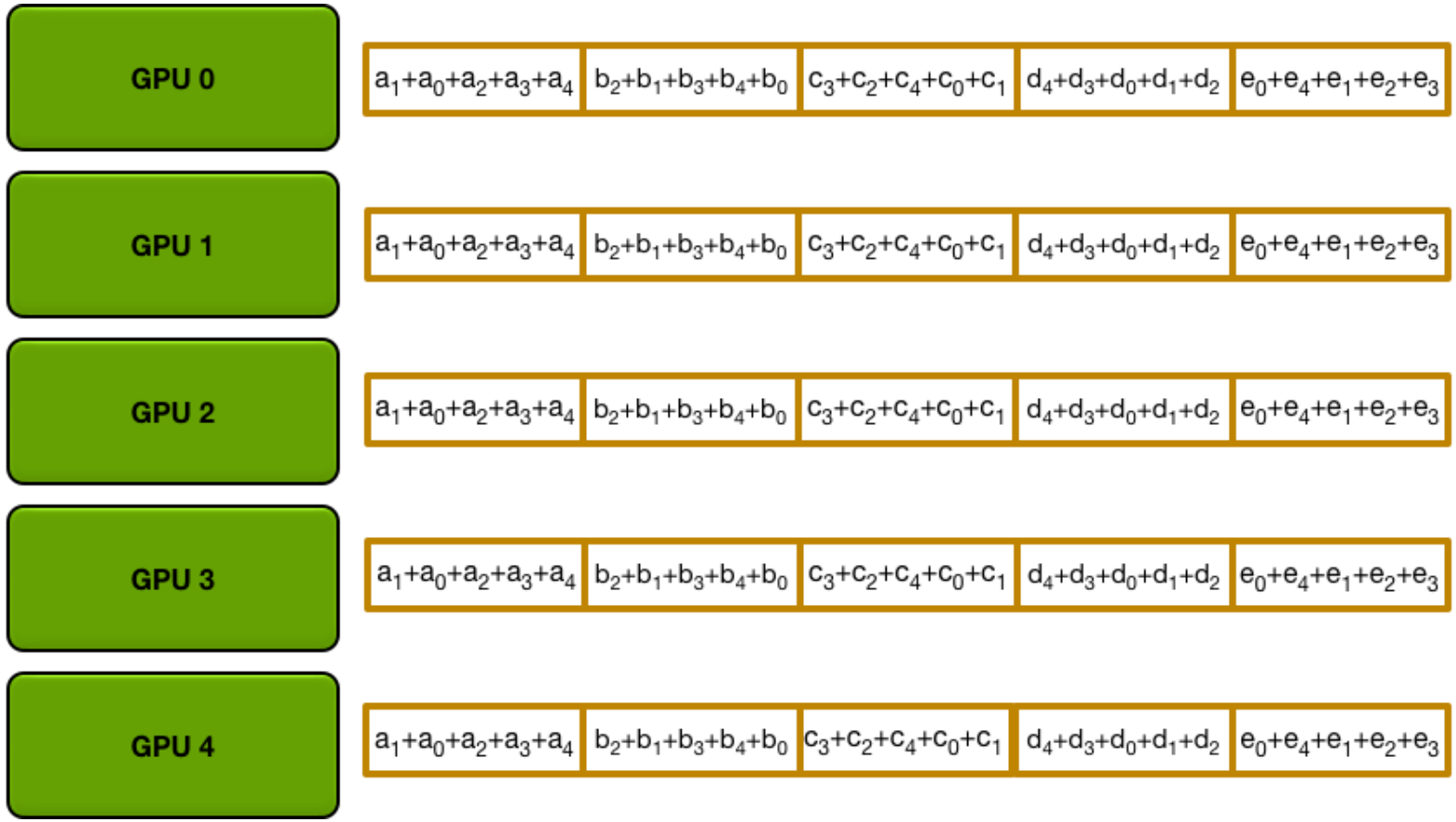
Allgather data transfer (iteration 3)

Phase 2: Allgather



Allgather data transfer (iteration 4)

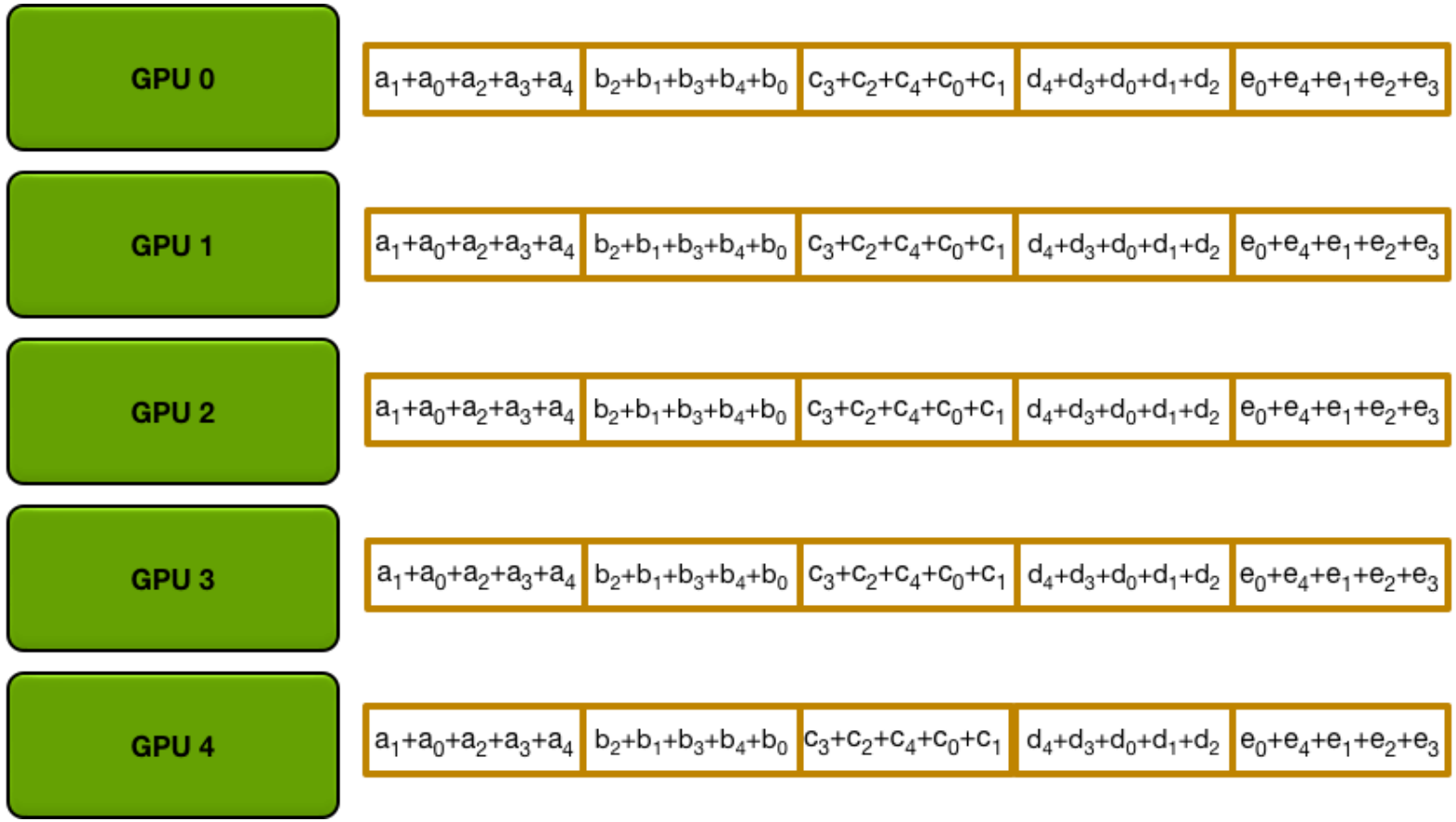
Phase 2: Allgather



Question: What is the time complexity of all-gather?

Final state after allgather transfer

Phase 2: Allgather



Question: What is the time complexity of all-gather?

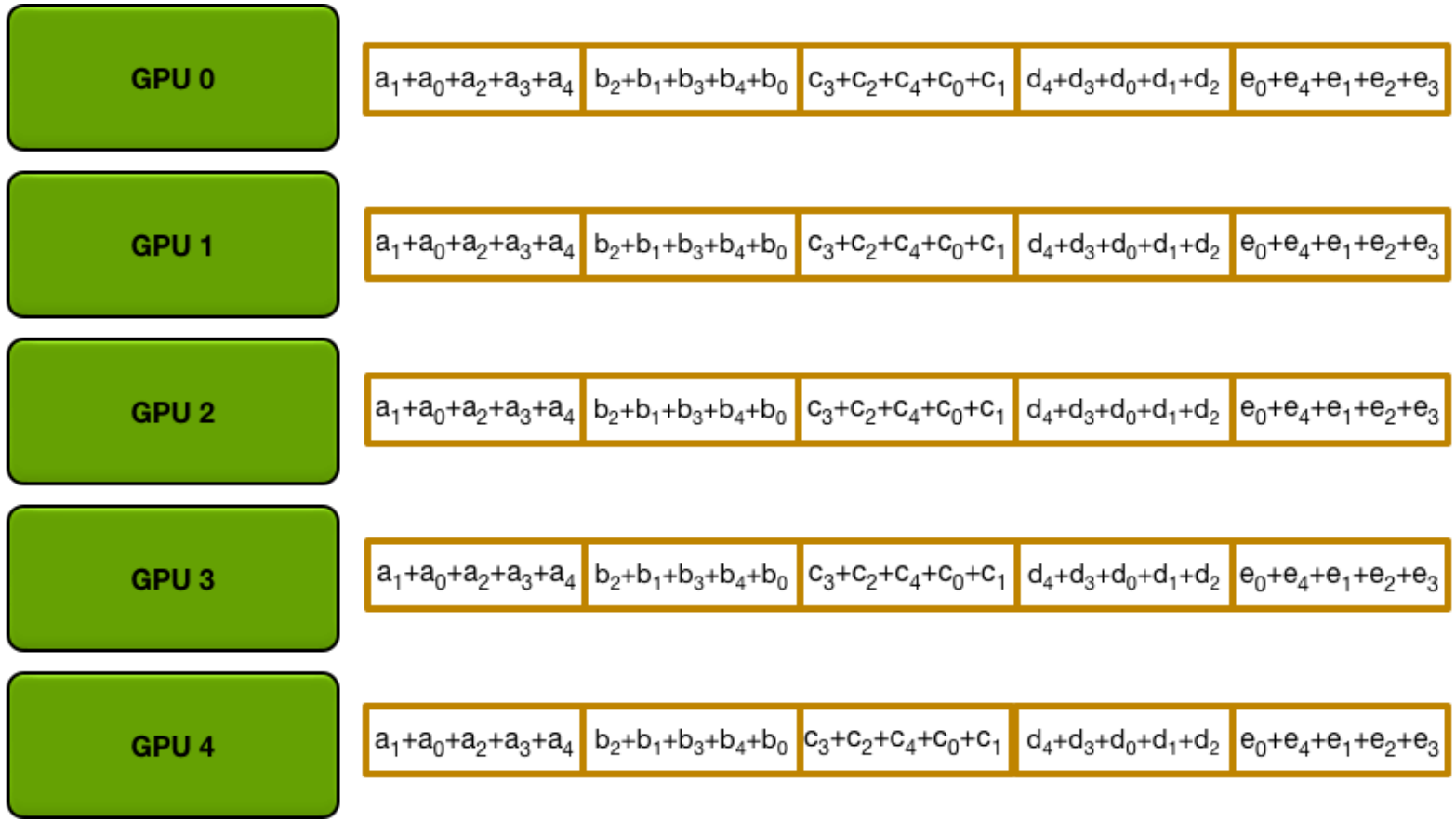
- N: Size of data
- P: # of workers/processes

$O(N/P * (P - 1))$

Chunk size

Final state after allgather transfer

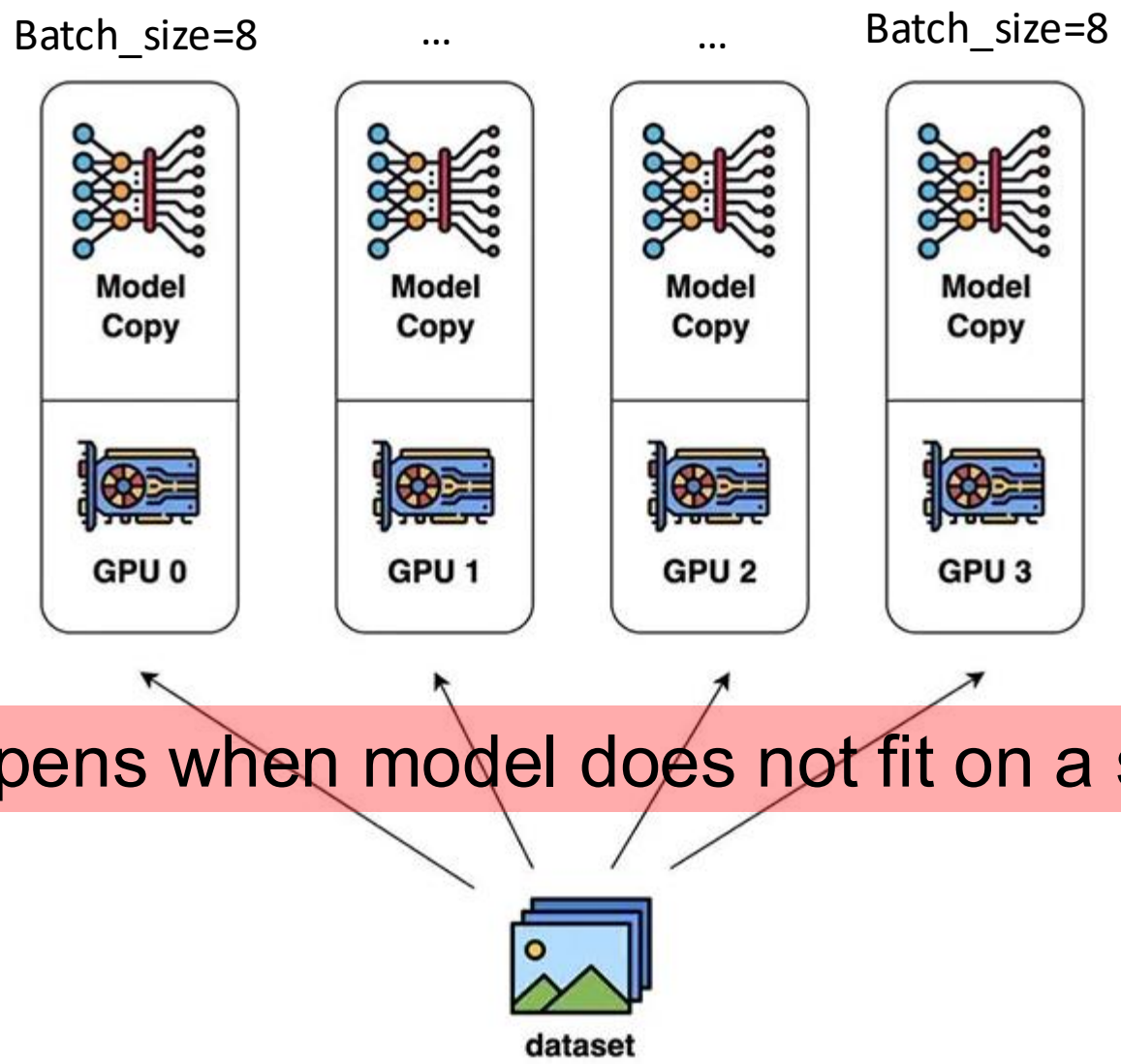
Phase 2: Allgather



First phase: N/P * (P – 1)
Second phase: N/P * (P -1)
Total: 2N (P – 1) / P, largely independent of P

Final state after allgather transfer

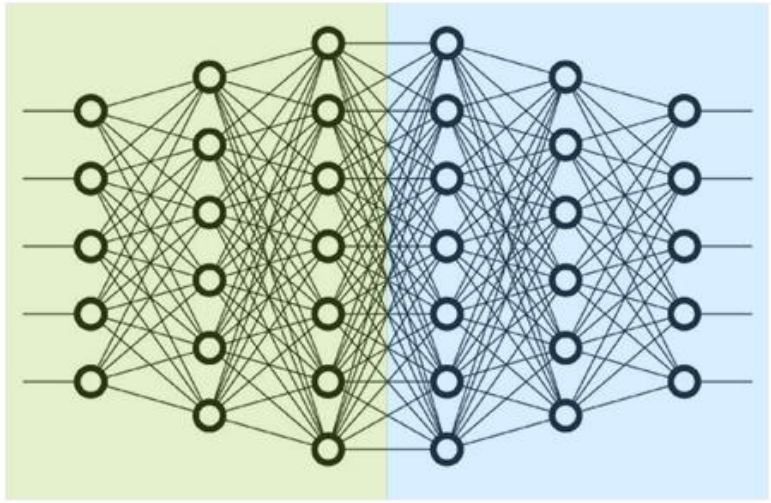
Data Parallelism (DP)



What happens when model does not fit on a single GPU?

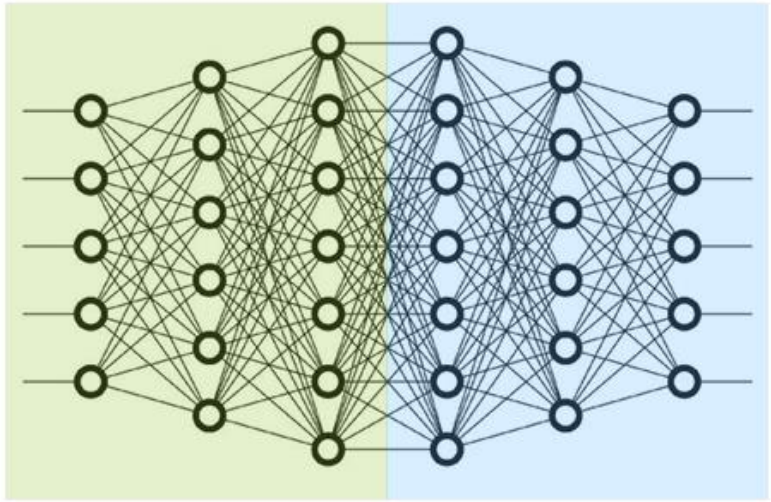
Pipeline Parallelism and Tensor Parallelism

Inter-layer (Pipeline) parallelism
- Split sets of layers across multiple workers
- Layer 0, 1, 2 and layer 3, 4, 5 are on different devices

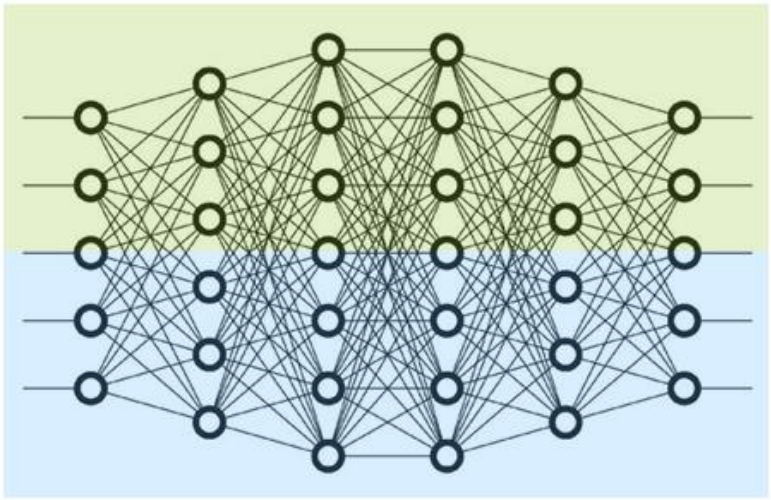


Pipeline Parallelism and Tensor Parallelism

Inter-layer (Pipeline) parallelism
- Split sets of layers across multiple workers
- Layer 0, 1, 2 and layer 3, 4, 5 are on different devices



Intra-layer (Tensor) parallelism
- Split individual layers across multiple workers
- Both devices compute different parts of layer 0, 1, 2, 3, 4, 5



Tensor Parallel: Parallel GEMM (General Matrix Multiplications)

Row-wise sharding (of A):

$$\begin{pmatrix} X_0 \\ X_2 \end{pmatrix} \begin{pmatrix} X_1 \\ X_3 \end{pmatrix} \begin{pmatrix} A_0 & A_1 \\ A_2 & A_3 \end{pmatrix} = \begin{pmatrix} X_0 \\ X_2 \end{pmatrix} \begin{pmatrix} A_0 & A_1 \end{pmatrix} + \begin{pmatrix} X_1 \\ X_3 \end{pmatrix} \begin{pmatrix} A_2 & A_3 \end{pmatrix}$$

$$= \underbrace{X_A A_A + X_B A_B}_{\text{Summation}}$$

Tensor Parallel: Parallel GEMM (General Matrix Multiplications)

Row-wise sharding (of A):

$$\begin{pmatrix} X_0 \\ X_2 \end{pmatrix} \begin{pmatrix} X_1 \\ X_3 \end{pmatrix} \begin{pmatrix} A_0 & A_1 \\ A_2 & A_3 \end{pmatrix} = \begin{pmatrix} X_0 \\ X_2 \end{pmatrix} \begin{pmatrix} A_0 & A_1 \end{pmatrix} + \begin{pmatrix} X_1 \\ X_3 \end{pmatrix} \begin{pmatrix} A_2 & A_3 \end{pmatrix}$$

$$= \underbrace{X_A A_A + X_B A_B}_{\text{Summation}}$$

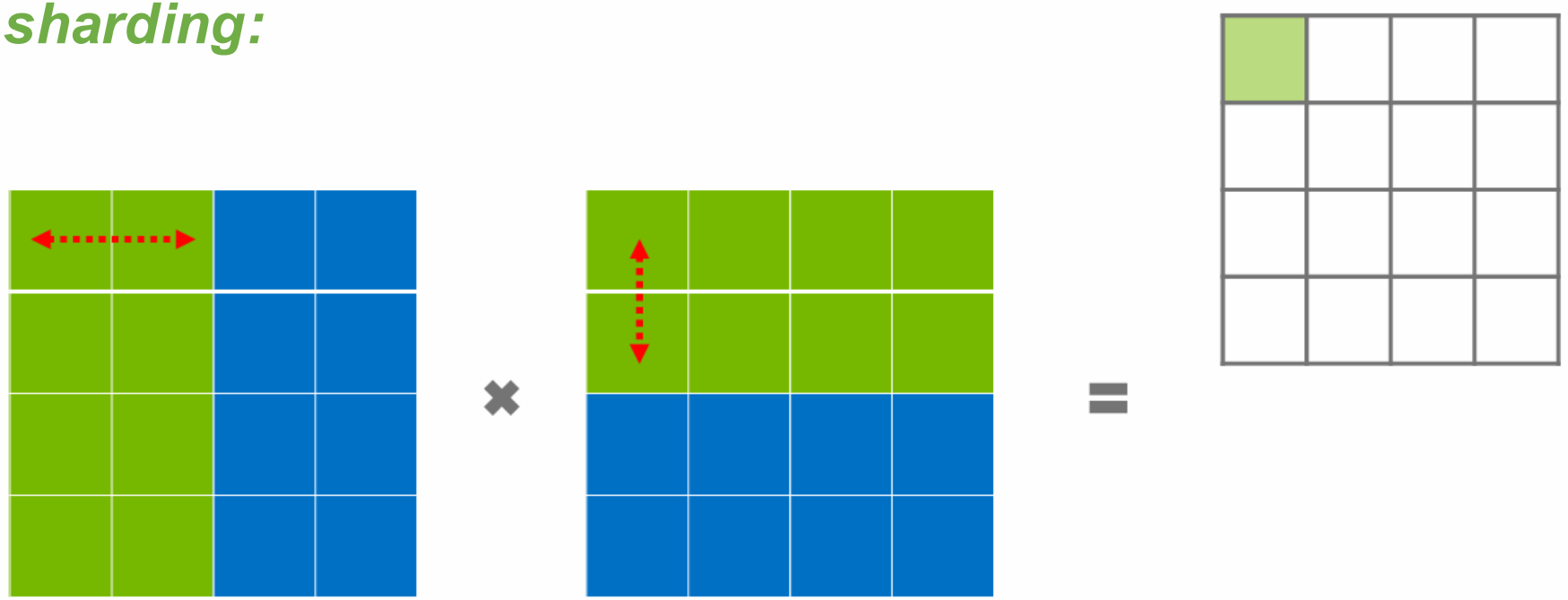
Column-wise sharding (of A):

$$\begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_0 \\ A_2 \end{pmatrix} \begin{pmatrix} A_1 \\ A_3 \end{pmatrix} = \left(\begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_0 \\ A_2 \end{pmatrix} \quad \begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_1 \\ A_3 \end{pmatrix} \right)$$

$$= \underbrace{\begin{pmatrix} XA_A & XA_B \end{pmatrix}}_{\text{Concatenation}}$$

Tensor Parallel: Parallel GEMM (General Matrix Multiplications)

Row-wise sharding:

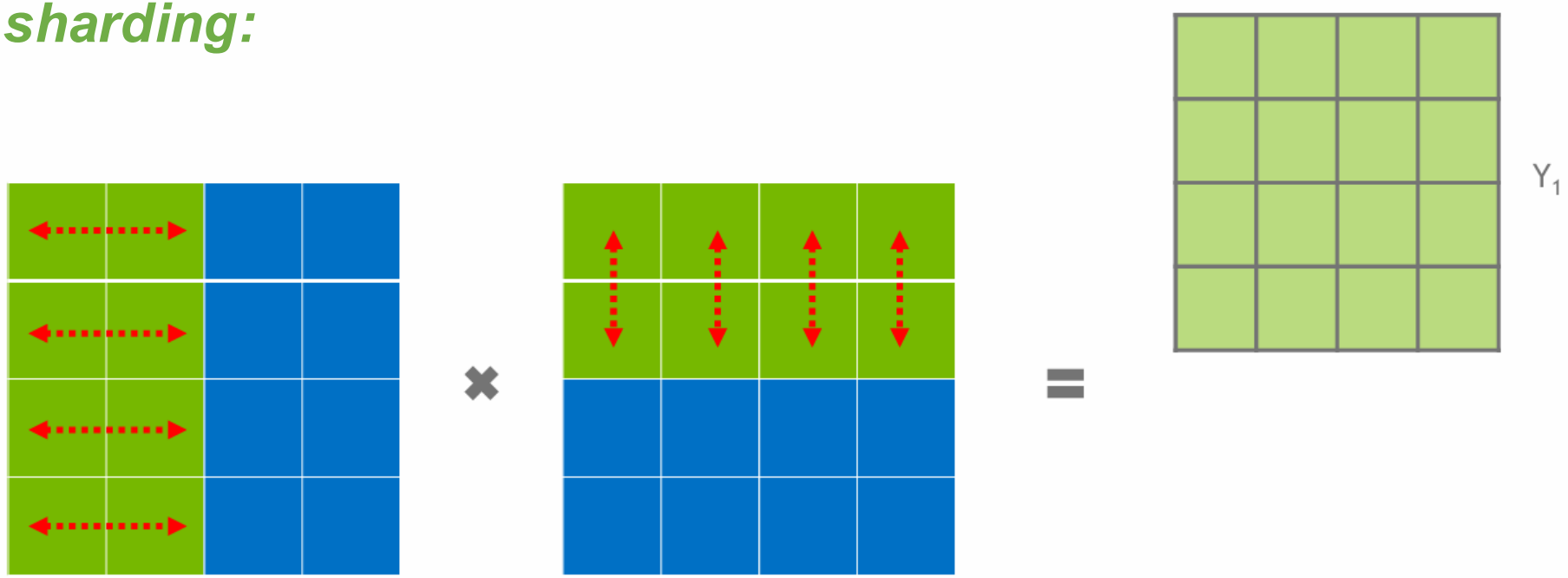


$$X = [X_1, X_2] \qquad A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix}$$

$$Y = Y_1 + Y_2$$

Tensor Parallel: Parallel GEMM (General Matrix Multiplications)

Row-wise sharding:

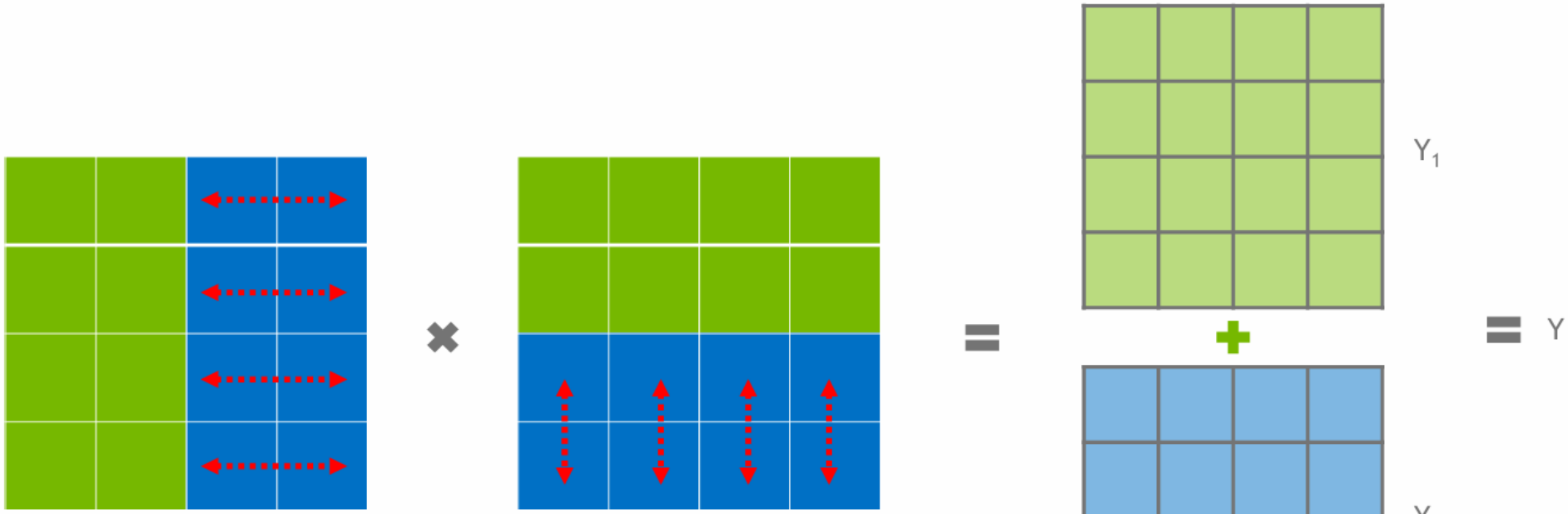


$$X = [X_1, X_2] \qquad A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix}$$

$$Y = Y_1 + Y_2$$

Tensor Parallel: Parallel GEMM (General Matrix Multiplications)

Row-wise sharding:

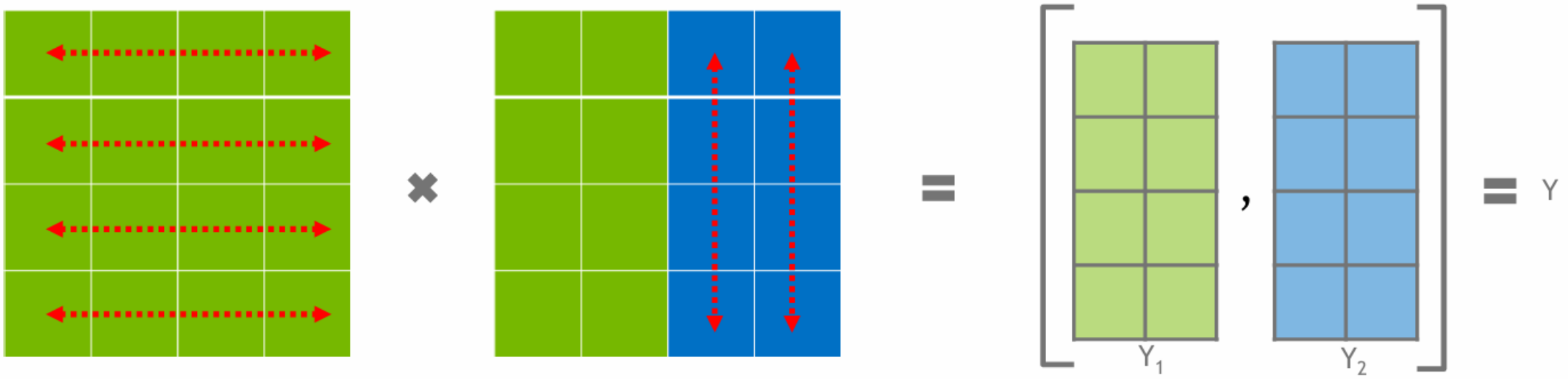


$$X = [X_1, X_2] \qquad A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix}$$

$$Y = Y_1 + Y_2$$

Tensor Parallel: Parallel GEMM (General Matrix Multiplications)

Column-wise sharding:



X $\quad A = [A_1, A_2]$ $\quad Y = [Y_1, Y_2]$

Tensor Parallel: Parallel GEMM (General Matrix Multiplications)

Row-wise sharding:

$$\begin{pmatrix} X_0 \\ X_2 \end{pmatrix} \begin{pmatrix} X_1 \\ X_3 \end{pmatrix} \begin{pmatrix} A_0 & A_1 \\ A_2 & A_3 \end{pmatrix} = \begin{pmatrix} X_0 \\ X_2 \end{pmatrix} \begin{pmatrix} A_0 & A_1 \end{pmatrix} + \begin{pmatrix} X_1 \\ X_3 \end{pmatrix} \begin{pmatrix} A_2 & A_3 \end{pmatrix}$$

$$= \underbrace{X_A A_A + X_B A_B}_{\text{Summation}}$$

Column-wise sharding:

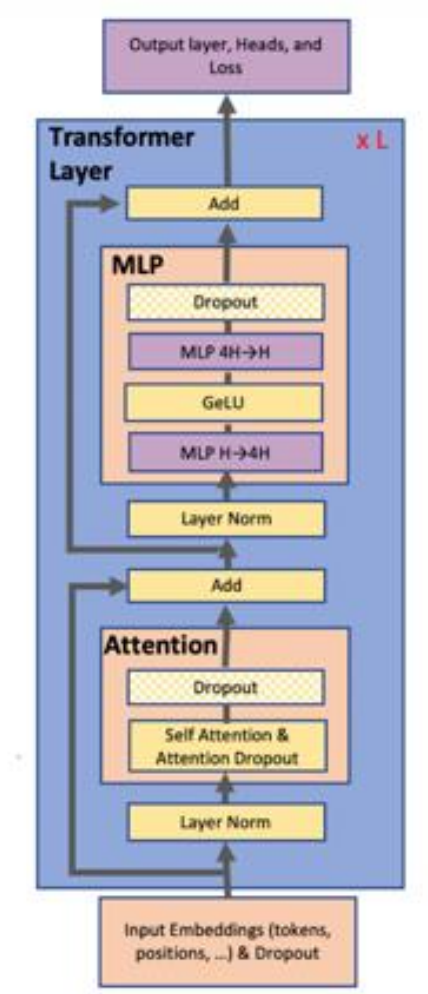
$$\begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_0 \\ A_2 \end{pmatrix} \begin{pmatrix} A_1 \\ A_3 \end{pmatrix} = \left(\begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_0 \\ A_2 \end{pmatrix} \quad \begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_1 \\ A_3 \end{pmatrix} \right)$$

$$= \underbrace{\begin{pmatrix} XA_A & XA_B \end{pmatrix}}_{\text{Concatenation}}$$

Column- or row-wise highly depends on tensor operations

Walk Through Transformers

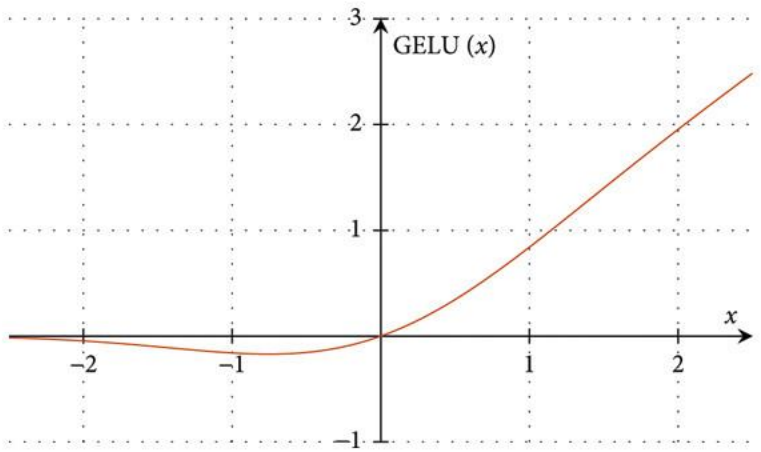
- Transformer layer
  - Self-attention block
  - Two-layer MLP
- Targets:
  - Reduce synchronization cost
  - Simple to implement with a few highly optimized collectives NCCL provides



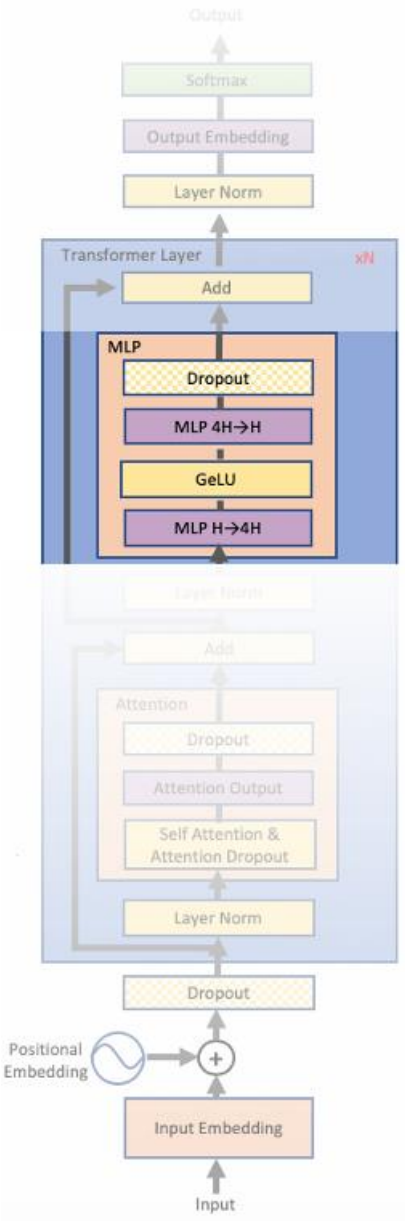
Tensor Parallelism: MLP

- MLP:

$$Y = \mathrm{GeLU}(XA)$$
$$Z = \mathrm{Dropout}(YB)$$



Dropout (YB):
Randomly drop a certain percentage of values to prevent overfitting



Tensor Parallelism: MLP

- MLP:

$$Y = \mathrm{GeLU}(XA)$$
$$Z = \mathrm{Dropout}(YB)$$

- Approach 1: split X column-wise and A row-wise

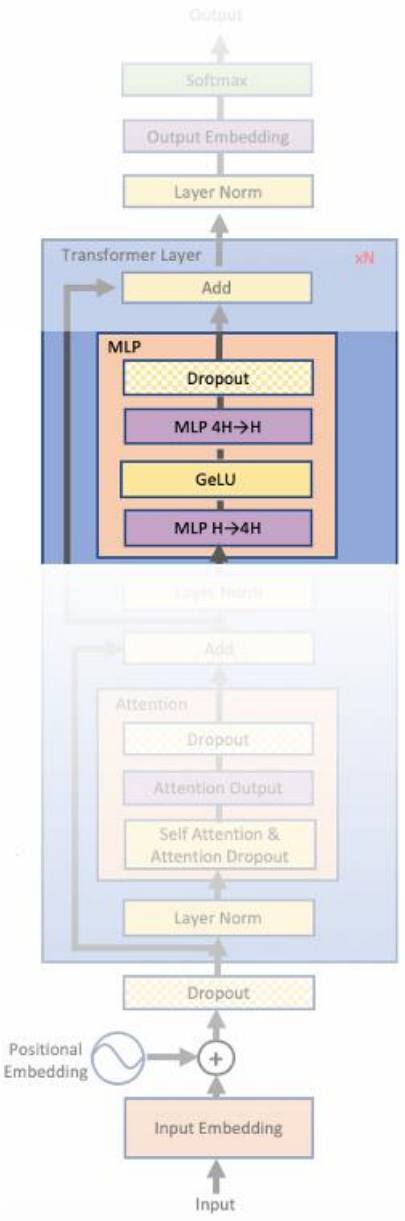
$$X = [X_1, X_2] \quad A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix} \longrightarrow Y = \mathrm{GeLU}(X_1A_1 + X_2A_2)$$

GeLU of sums != sum of GeLUs

$$\begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_0 & A_1 \\ A_2 & A_3 \end{pmatrix} = \begin{pmatrix} X_0 \\ X_2 \end{pmatrix} \begin{pmatrix} A_0 & A_1 \end{pmatrix} + \begin{pmatrix} X_1 \\ X_3 \end{pmatrix} \begin{pmatrix} A_2 & A_3 \end{pmatrix}$$

$$= X_A A_A + X_B A_B$$

Summation



Tensor Parallelism: MLP

- MLP:

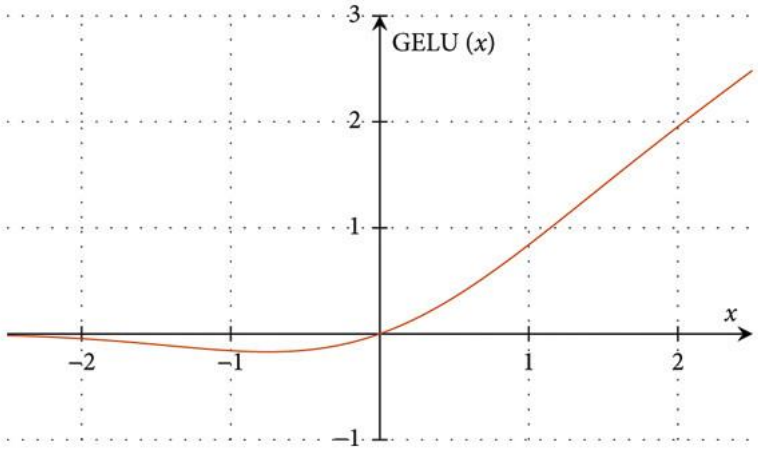
$$Y = \mathrm{GeLU}(XA)$$
$$Z = \mathrm{Dropout}(YB)$$

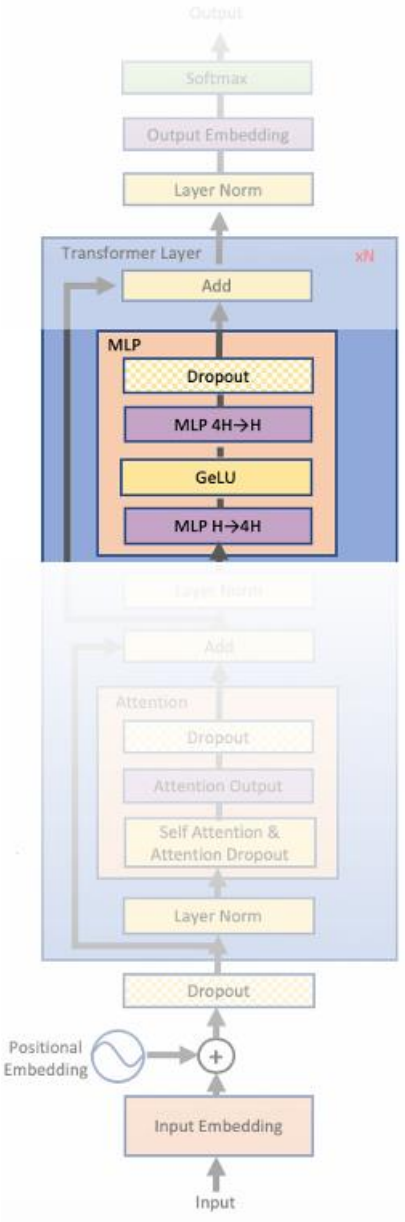
- Approach 1: split X column-wise and A row-wise

$$X = [X_1, X_2] \quad A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix} \longrightarrow Y = \mathrm{GeLU}(X_1A_1 + X_2A_2)$$

Requires AllReduce before GeLU

GeLU of sums != sum of GeLUs





Tensor Parallelism: MLP

- MLP:

$$Y = \mathrm{GeLU}(XA)$$
$$Z = \mathrm{Dropout}(YB)$$

- Approach 1: split X column-wise and A row-wise

$$X = [X_1, X_2] \quad A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix} \longrightarrow Y = \mathrm{GeLU}(X_1A_1 + X_2A_2)$$

  - Requires synchronization before GeLU

GeLU of sums != sum of GeLUs

- Approach 2: split A column-wise

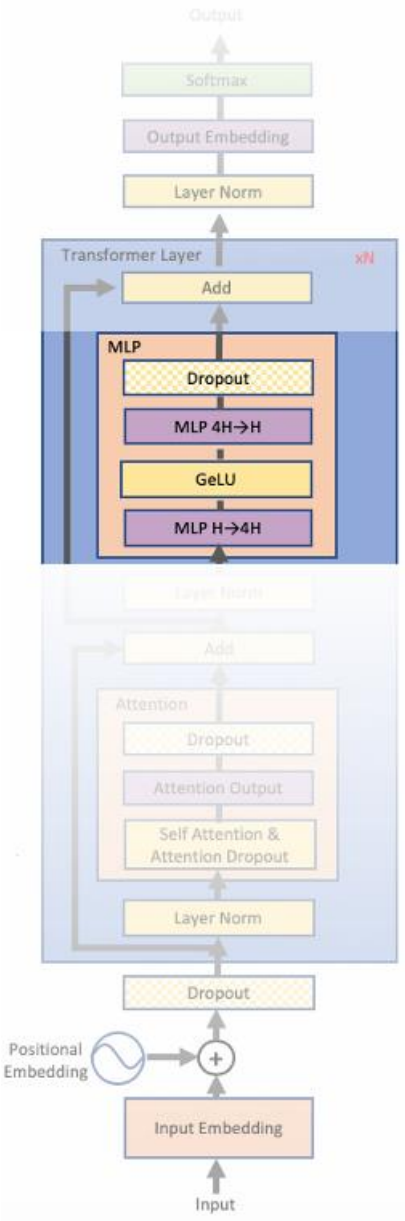
$$A = [A_1, A_2] \longrightarrow [Y_1, Y_2] = [\mathrm{GeLU}(XA_1), \mathrm{GeLU}(XA_2)]$$

  - No synchronization necessary

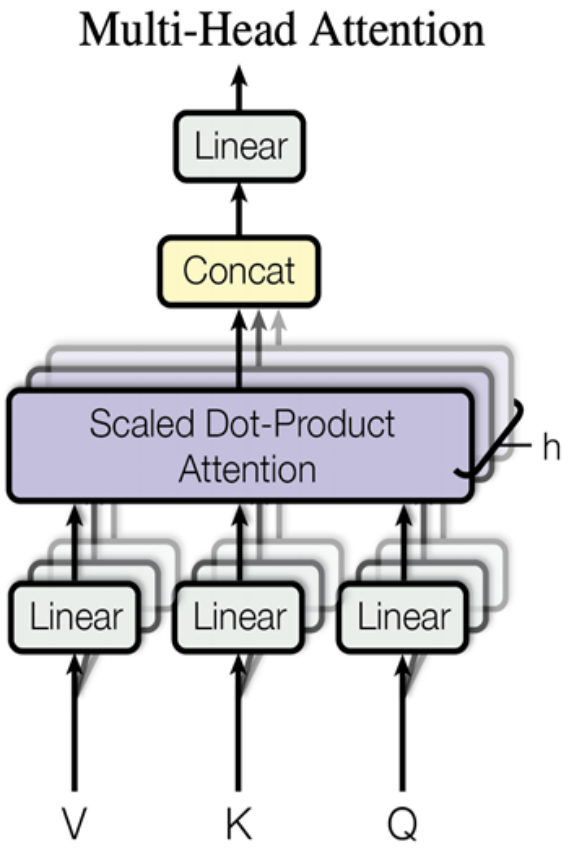
Can run in parallel now!

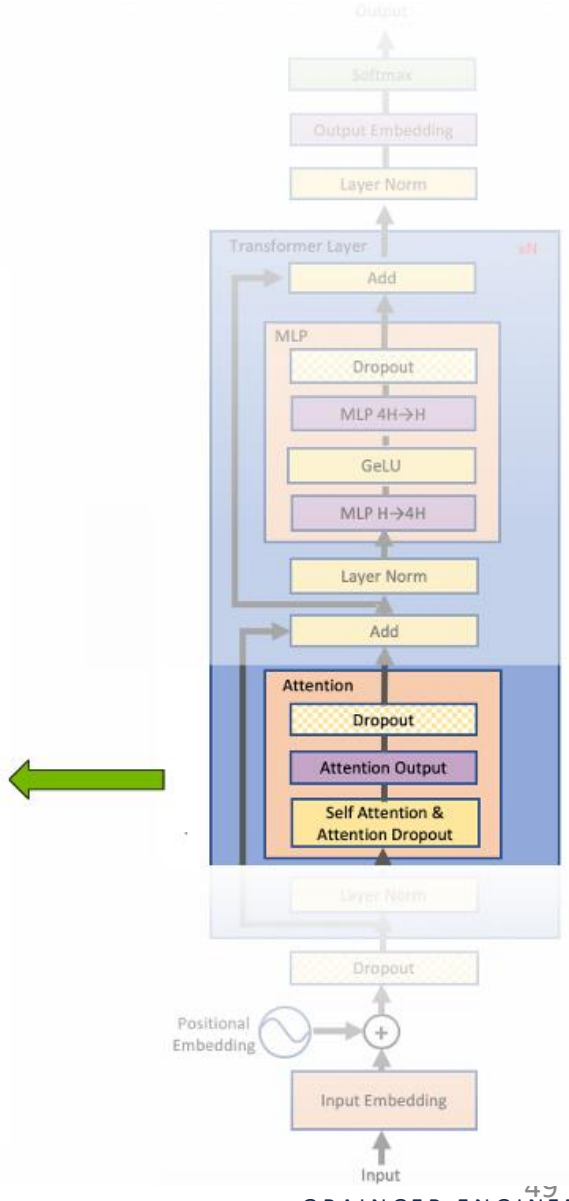
$$\begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_0 & A_1 \\ A_2 & A_3 \end{pmatrix} = \left(\begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_0 \\ A_2 \end{pmatrix} \quad \begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_1 \\ A_3 \end{pmatrix} \right)$$

$$= \underbrace{(XA_A \quad XA_B)}_{\text{Concatenation}}$$

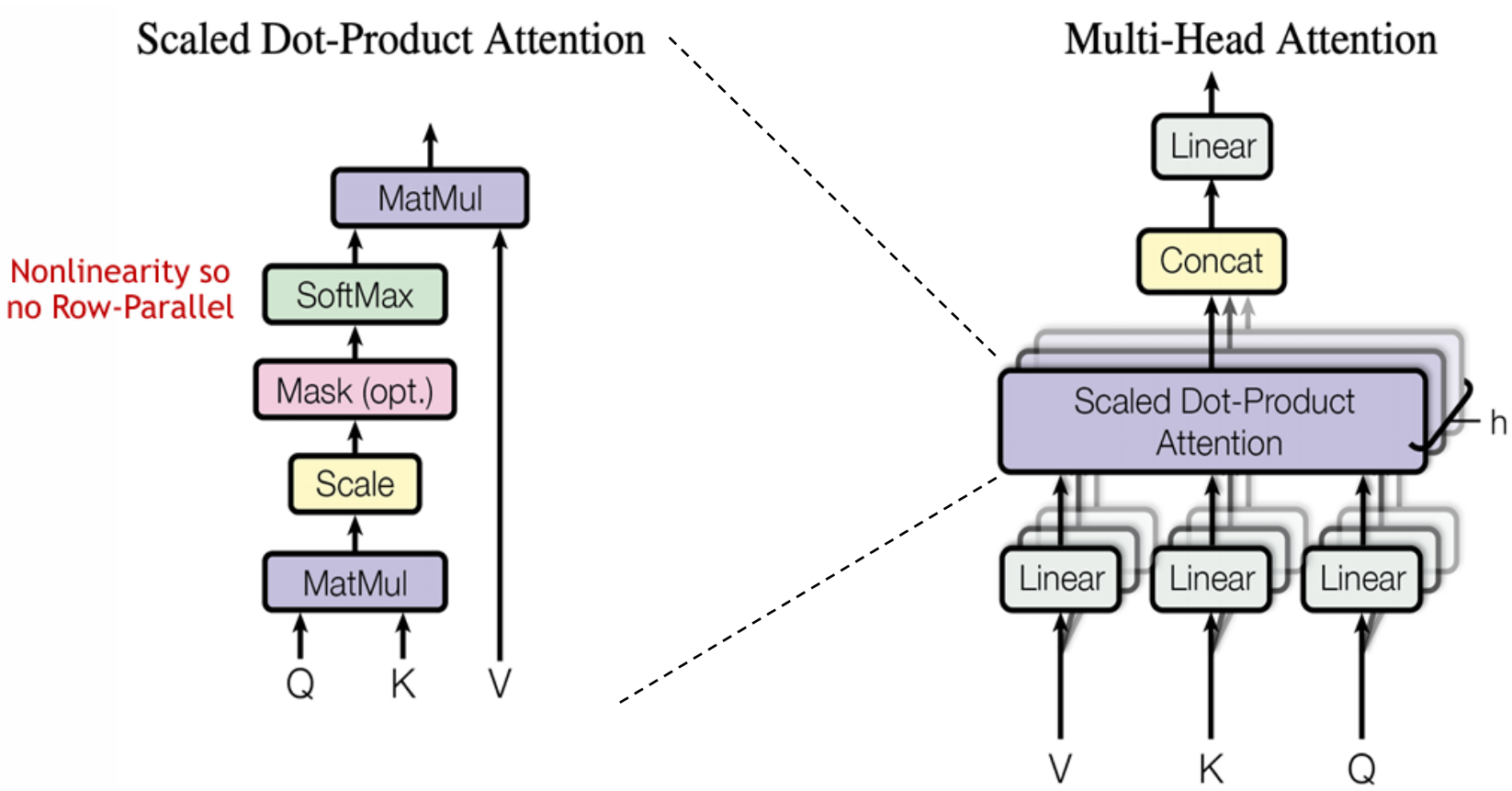


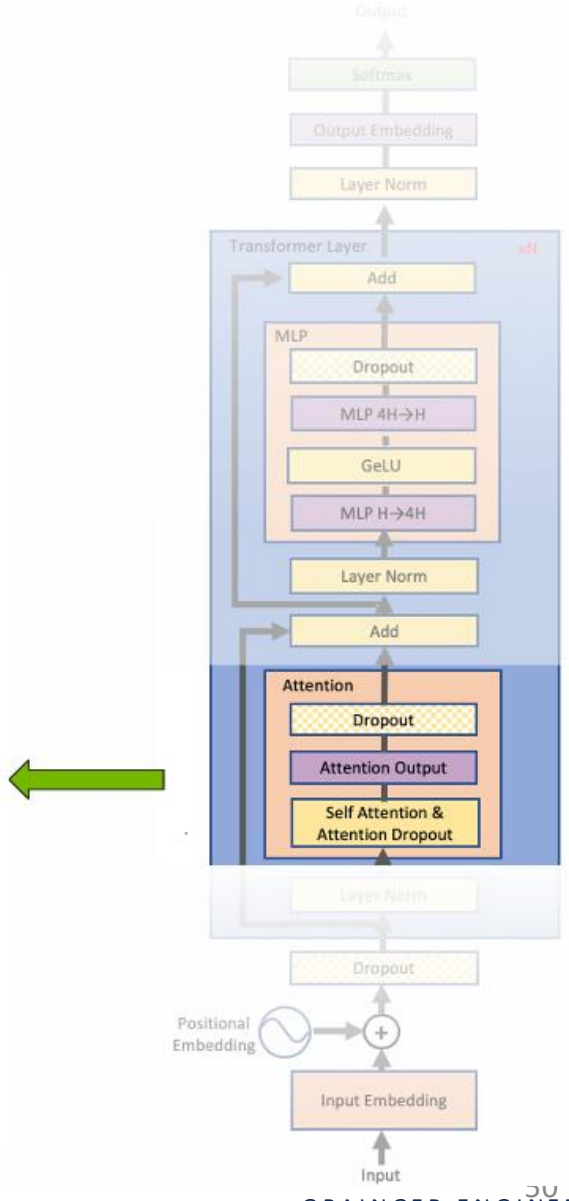
Tensor Parallelism: Self-Attention



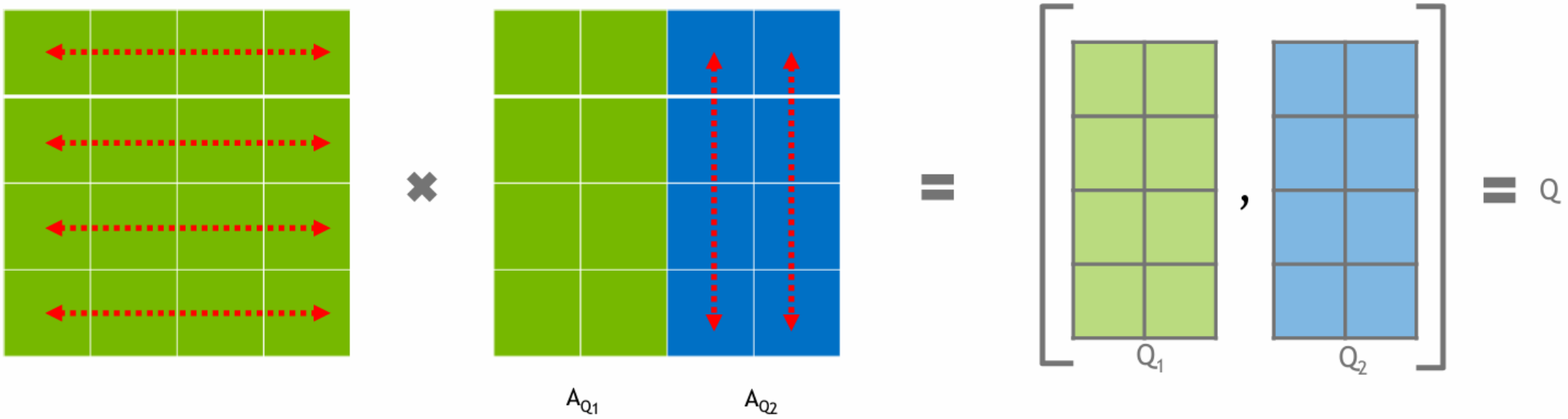


Tensor Parallelism: Self-Attention





Tensor Parallelism: Self-Attention



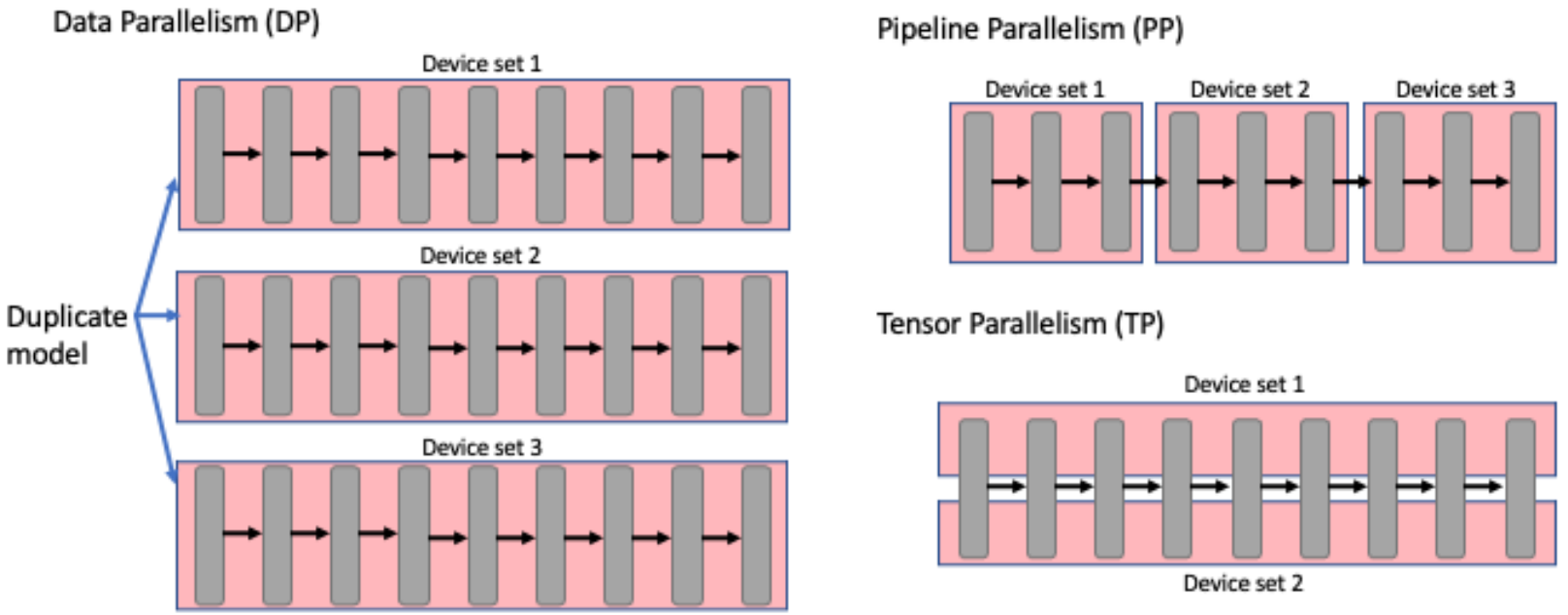
X $A = [A_1, A_2]$ $Y = [Y_1, Y_2]$

Attention heads can be parallelized with Column Parallel GEMMs (ex. Query head 1 (Q_1) and Query head 2 (Q_2))

Question: What if we have more GPUs than the number of heads?

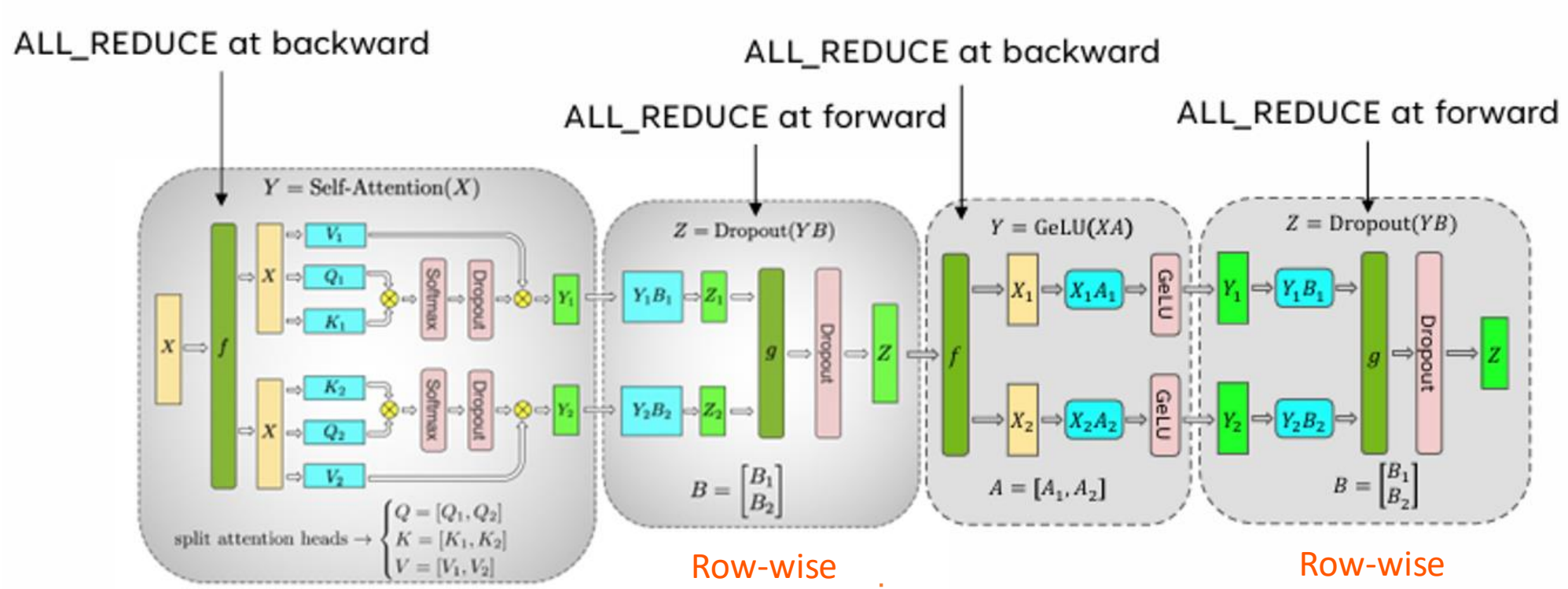
Summary

- Data parallelism assumes hosting the full model within a unit
- Tensor parallelism (TP): split matrix row-wise or column-wise
- Reduce synchronization cost is crucial as TP is communication-intensive
- How to automate parallelism planning? (Feb 11)



Tensor Parallelism: Communication Cost

- 4 total communication operations in 1 forward + backward pass



$$\begin{pmatrix} X_0 \\ X_2 \end{pmatrix} \begin{pmatrix} X_1 \\ X_3 \end{pmatrix} \begin{pmatrix} A_0 & A_1 \\ A_2 & A_3 \end{pmatrix} = \begin{pmatrix} X_0 \\ X_2 \end{pmatrix} (A_0 \quad A_1) + \begin{pmatrix} X_1 \\ X_3 \end{pmatrix} (A_2 \quad A_3)$$

$$= \underbrace{X_A A_A + X_B A_B}_{\text{Summation}}$$